

Doc rules (copied from ICC doc :))

1. Everyone is free to read it!
2. The answers for each question can consist of 3 parts:
 - a. the links to resources where it is explained well (**please make them as laconic as you can**)
 - b. the descriptions/ideas/explanations that you come up with regarding these questions. contrary to the previous one, **please make them as detailed as you can:) and put the references where needed (from where did you conclude this or that thing)**
 - c. the notes about what is still unclear/uncovered within this question (we'll probably make them closer to the exam date)
3. Everyone is free (and supposed to...) write the answers here (a-c type from p.2)
 - a. if there is some explanation, but you see that it's unclear or you can explain it better, feel free to add your explanation!
 - b. **Please sign who has written what on each note, so that we can ask a direct question to the author of it!**
4. If you have some precise questions about what is written in the answers, please make a comment in the doc, do not clutter up the answers!
5. Use the table of contents to navigate; the document is loong!
6. I [Kate] reordered the questions so that they are in chronological order (as they are appearing in the lectures). If there are mistakes in this order, feel free to swap the questions :)

Exam rules

- You should know all theory presented at the lecture: definitions of data structures and operations on them, all theorems and proofs thereof.
- The exam is oral with written preparation. You will get **one major** and **one minor** question from the following list.
- [Original list of questions](#)
- [Rules for distant exams](#)

Materials

- [videos + board images from last year](#)
- [structured lecture notes as text](#)
- [notes of this year](#)

Content

Done ☒ In progress ☐

Please mark the status of the question here so that people easily know on which questions to work. But comments and additions to *done* questions are also appreciated.

1. ☒ [Minor] [Describe a flexible array with growing and shrinking. Analyze its amortized complexity.](#)
2. ☒ ☐ [Minor] [Define the lazily balanced trees BB\[alpha\]. Analyze their amortized complexity.](#)
3. ☒ [Major] [Define the Splay tree. State and prove the theorem on amortized complexity of the Splay operation.](#)
4. ☒ [Minor] [Design operations Find, Insert, and Delete on a Splay tree. Analyze their amortized complexity.](#)
5. ☒ [Major] [Define the \(a,b\)-tree. Describe operations Find, Insert, and Delete. Analyze their complexity in the worst case.](#)
6. ☒ [Minor] [State and prove the theorem on amortized complexity on Insert and Delete on \(a,2a-1\)-trees and \(a,2a\)-trees.](#)
7. ☒ [Minor] [Analyze k-way Mergesort in the cache-aware model. Which is the optimum value of k?](#)
8. ☒ [Major] [Formulate a cache-oblivious algorithm for transposition of a square matrix. Analyze its time complexity and I/O complexity.](#)
9. ☒ [Minor] [State and prove the Sleator-Tarjan theorem on competitiveness of LRU.](#)
10. ☒ [Major] [Describe hashing with chains and analyze its complexity. Define c-universal and k-independent systems of hash functions and provide constructions of such systems.](#)
11. ☒ [Minor] [Describe a system of linear hash functions. Prove that it is a 2-independent system from \$Z_p\$ to \$\[m\]\$.](#)
12. ☒ [Minor] [Construct a k-independent system of hash functions from \$Z_p\$ to \$\[m\]\$.](#)
13. ☒ [Minor] [Construct a 2-independent system of hash functions for hashing of strings of length at most L over an alphabet \[a\] to a set of buckets \[m\].](#)
14. ☒ [Minor] [Describe a system of hash functions based on scalar products. Prove that it is a 1-universal system from \$Z_p^k\$ to \$Z_{p^2}\$.](#)
15. ☒ [Minor] [Describe the cuckoo hashing and state the theorem on its complexity \(without proof\).](#)
16. ☒ [Minor] [Describe hashing with linear probing and give overview of results on its complexity.](#)
17. ☒ [Minor] [Describe and analyze the Bloom filter.](#)
18. ☒ [Major] [Define suffix arrays and LCP arrays. Describe and analyze algorithms for their construction.](#)
19. ☒ [Minor] [Show how to use suffix array and LCP array for finding the longest common substring of two strings.](#)
20. ☒ [Minor] [Show how to perform 1-dimensional range queries on binary search trees.](#)
21. ☒ [Minor] [Define k-d trees and show that they require \$\Omega\(\sqrt{n}\)\$ time per 2-d range query.](#)
22. ☒ [Major] [Define multi-dimensional range trees. Analyze time and space complexity of construction and range queries.](#)
23. ☒ [Minor] [Describe parallel \(a,b\)-trees with the use of locks.](#)
24. ☒ [Minor] [Describe a lock-free stack including the memory management.](#)

Prerequisites

Amortized complexity

Kate, [lecture notes on potential](#)

amortized cost A_i — our opinion on how much the operation contributes to the total execution time.

We can choose it arbitrarily as long as the sum of all amortized costs is an upper bound on the sum of all real costs.

Difference between amortized and average-case complexity:

- averages can be computed over all possible inputs or over all possible random bits generated in a randomized algorithm, but they do not promise anything about a specific computation on a specific input.
- amortized complexity guarantees an upper bound on the total execution time, but it does not reveal anything about distribution of this time between individual operations.

Potential

Kate, [lecture notes](#)

Each operation is characterized by:

real cost R_i — the time spent by the machine on performing the operation.

we often bound it by the worst-case cost for simplicity.

amortized cost A_i — our opinion on how much the operation contributes to the total execution time.

We can choose it arbitrarily as long as the sum of all amortized costs is an upper bound on the sum of all real costs.

Execution of the first i operations takes real cost $R_1 + \dots + R_i$.

But we claimed that it will take amortized cost $A_1 + \dots + A_i$.

Potential of the data structure: the difference between real and amortized costs

It is a kind of a balance of a “bank account” used to save time for the future.

$$\phi_i = \left(\sum_{j=1}^i A_j \right) - \left(\sum_{j=1}^i R_j \right)$$

Before we perform the first operation, we naturally have $\Phi_0 = 0$.

At every moment, we have $\Phi_i \geq 0$ as we can never owe time.

Potential difference:

$$\Delta\Phi_i = \Phi_i - \Phi_{i-1} \text{ and } \Delta\Phi_i = A_i - R_i$$

It tells us if the i -th operation saves time ($\Delta\Phi_i > 0$)
or it spends time saved in the past ($\Delta\Phi_i < 0$).

Use an inverse approach:

first we choose the potential according to the state of the structure
then we use it to calculate the amortized complexity

⇒ We can calculate the amortized cost like this: $A_i = R_i + \Delta\Phi_i$ - the amortized cost is the sum of the real cost and the potential difference.

⇒ The sum of all amortized costs:

$$\sum_{i=1}^m A_i = \sum_{i=1}^m (R_i + \Phi_i - \Phi_{i-1}) = \left(\sum_{i=1}^m R_i \right) + \Phi_m - \Phi_0.$$

telescoping — each Φ_i except for the first one and the last one is added once and subtracted once, so it disappears.

As long as $\Phi_m \geq \Phi_0$, $(\sum_{j=1}^i A_j) \geq (\sum_{j=1}^i R_j)$ as we wanted.

Algorithm of working with the potentials:

- choose an appropriate potential function Φ
 - depends on the current state of the structure and possibly also on its history.
 - There is no universal recipe for choosing it, but we usually want the potential to increase when we get close to a costly operation.
- try to prove that $\Phi_0 \leq \Phi_m$ (we “do not owe time”).
 - This is easy if our potential counts occurrences of something in the structure — this makes it always non-negative and zero at the start.
 - If the inequality does not hold, we must compensate for the “borrowed time” by assigning non-zero amortized cost to data structure initialization or cleanup.
- establish the amortized costs of operations from their real costs and the potential differences: $A_i = R_i + \Phi_i - \Phi_{i-1}$.

Basically, when using the potentials, we need to choose the potential function wisely.

External memory model aka I/O model

Kate, [lecture notes](#)

Used for working with data on disks, which do not fit in the machine’s main memory.

Two types of memory:

External memory of potentially infinite size, organized in blocks of size B — since we are going to study asymptotics, units do not matter, so we will measure all sizes in arbitrary items.

Internal memory for M items, also organized in B -item blocks.

All computations are performed in the internal memory similar to the Random Access Machine.

External memory can be accessed only indirectly:

a block of data has to be loaded to the internal memory first
if it is modified, it has to be written back to the external memory later.

I/O complexity of algorithms: the maximum number of input/output operations (reads and writes of blocks) performed by the algorithm for a given size of input.

Calculation of I/O complexity is often reduced to just counting the number of block reads, since the number of writes is bounded by the number of reads in most cases.

- When intermediate results are written, they are read later.
- Unless the final output is asymptotically larger than the input, the number of writes needed to create the output is bounded by the number of reads needed to obtain the input.

Important remark: very often I/O complexities will include $+1$ inside.

When we divide data of length T into blocks of size B , the number of blocks needed to be read is T/B . We need this $+1$ because $T/B < 1$ if all items fit into one block, and this 1 plays a role of rounding here. So we cannot remove it from O !

Also in caching models we cannot replace the O by Θ since some of the blocks could be already present in the cache and the lower bound would be constant in this case.

Cache models (cache-aware and cache-oblivious)

Kate, [lecture notes](#)

Machines can accelerate access to main memory by using a cache.

External memory: machine's main memory

Internal memory: cache

BUT difference from I/O model: block transfers between main memory and the cache are controlled by the machine and not by the program itself

The program accesses data in the main memory.

Cache controller - the part of the machine which handles transfers of the particular blocks between the main memory and the cache automatically.

- works optimally, i.e. it decides which blocks should be kept in the cache and which should be evicted in a way minimizing the total number of I/O operations.
- unrealistic - an optimal cache controller needs to know the future accesses
- but there's [proof](#) that there exists a deterministic strategy which approximates this controller within a constant factor.

Cache-aware model - the algorithm knows the parameters of the cache:
the block size B

the cache size M
easier to implement algorithms for

Cache-oblivious model – the algorithm doesn't know the parameters of the cache.

But cache controller does and it can apparently estimate when the subproblems of the algorithm can be cached effectively

more useful

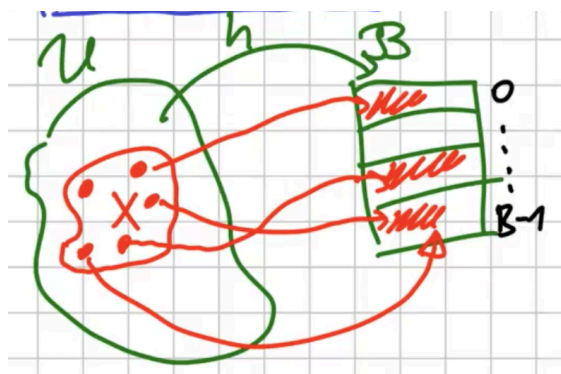
algorithms for cache-oblivious model can work efficiently on many machines with different cache architectures

if the machine uses a hierarchy of several caches, the algorithm can be shown to be optimal (within a constant factor) with respect to all caches simultaneously.

Hashing notation

Kate

- The universe $\mathcal{U}$.
Usually, the universe is a set of integers $\{0, \dots, U - 1\}$, which will be denoted by $[U]$.
- The set of buckets $\mathcal{B} = [m]$.
- The set $X \subset \mathcal{U}$ of n items stored in the data structure.
- Hash function $h : \mathcal{U} \rightarrow \mathcal{B}$.



[Optional] Fields

Andy

Sometimes the lectures refer to [fields](#), so if your knowledge of fields is rusty (or nonexistent), I think it is enough to know that it is a set with arithmetic operations (addition, multiplication) defined over it. An obvious example of an infinite field is the rational numbers. It mostly comes up in hashing, where we often have the set of integers modulo prime p , notated as $\mathbb{Z}_p$, which is a finite field because its operations are defined as described [here](#).

In a field, multiplication of a non-zero number by a uniformly random number yields a uniformly random number; similarly, adding a uniformly random number to any number yields a uniformly random result. This forms part of the intuition behind the proof of [this question](#).

Parallel computing and synchronization

Toni

All of the following things are not something you would write on the exam, but are needed to understand the answers to the exam questions

Background story

Modern computers use multiple processors (=cores) that are used for efficient use of computing power by running multiple processes and threads which run in parallel on different processors. We will use a simple theoretical model: Parallel RAM (PRAM), which consists of multiple RAMs and each instance is called a processor. Instructions of the machine access local memory of the current processor, but also the global memory which is shared among all processors. Computing directly on global memory is not allowed, so processors need to load the value they need, do something with it and write it back to the global memory. When running a program in parallel on multiple processors, the same code is sent to all of them, but the execution on each one may depend on some conditions in the code that contain processor ID (example: if processorID == 42: functionX() else: functionY())

Problems occur when multiple processes access the same cell of the global memory at the same time. If they want to read, we have no problem. But if at least one of them wants write a value there, it will have undefined behaviour - depending on who got there first. This is called [race condition](#), and it is unacceptable - we want programs that are independent on the random events like this. This is why we need locks and atomic operations.

Locks

Imagine a variable *cnt* in global memory and an increment operation *GlobalInc()*. This operation would look something like this:

GlobalInc()

$t \leftarrow cnt$

$t \leftarrow t + 1$

$cnt \leftarrow t$

Remember, we cannot directly compute on global memory so we load the value to a local variable *t*, we increment it and write it back. This works fine if we have one processor, but in the case of 2 processors, a race condition happens.

Lock (or *mutex*) is a synchronization primitive which helps in these situations. At any given moment, the mutex is either unlocked or locked. Two operations of mutex:

- Lock - If the mutex is unlocked, lock it. If it is locked, wait until it is unlocked by somebody else and then lock it.
- Unlock: If the mutex is locked, unlock it. If it is unlocked, crash (this shouldn't happen in a correct program)

Typically, each instance of a data structure has its own mutex and every operation on the data structure is wrapped in an Lock/Unlock pair. This guarantees that only one process is working with the data structure at any given moment. For us, these operations on the data structure appear to be atomic (not divisible into smaller suboperations - it either didn't start or it finished). But if we want to work with multiple instances of a data structure, problems arise and deadlock may occur. [Deadlock in lecture notes](#)

Lemma M for hashing families (modulo m to reduce counter-domain in hashing)

Jacob

This applies to Lemmas M, K and G: we sometimes have hashing families that map from some domain to $\mathbb{Z}_p$. However, we sometimes need to reduce that further to simply $[m]$. We can do this in various manners. This is what lemmas M, K and G address.

Lemma M basically tells us that we can do modulo m at the end of our usual computation from some domain to $\mathbb{Z}_p$.

Lemma M states:

Let $\mathcal{H}$ be a $(2, c)$ -independent family of functions from a universe $\mathcal{U}$ to $[r]$ and $m < r$. Then the family $\mathcal{H} \bmod m = \{h \bmod m \mid h \in \mathcal{H}\}$ is $2c$ -universal and $(2, 4c)$ -independent.

Lemma K for hashing families (extension of lemma M)

Lemma K generalizes the above for k -independent families. It states the following:

Let $\mathcal{H}$ be a (k, c) -independent family of functions from a universe $\mathcal{U}$ to $[r]$ and m integer such that $r \geq 2km$. Then the family $\mathcal{H} \bmod m = \{h \bmod m \mid h \in \mathcal{H}\}$ is $(k, 2c)$ -independent.

Lemma G for hashing families (reduce counter-domain by composition)

[Kate's comment] function composition is an operation $\circ$ that takes two functions f and g , and produces a function $h = g \circ f$ such that $h(x) = g(f(x))$

Let $\mathcal{F}$ be a c -universal family of functions from $\mathcal{U}$ to $[r]$. Let $\mathcal{G}$ be a $(2, d)$ -independent family of functions from $[r]$ to $[m]$. Then the family $\mathcal{H} = \mathcal{F} \circ \mathcal{G} = \{f \circ g \mid f \in \mathcal{F}, g \in \mathcal{G}\}$ is $(2, c')$ -independent for $c' = (cm/r + 1)d$.

The proof for this lemma is pretty concise in [the notes \(p. 68\)](#)

Corollary: The family $\mathcal{F} \circ \mathcal{G}$ is also $(2, c')$ -independent for $c' = (c + 1)d$.

This is strictly less powerful but pretty useful. We know this because a 2-independent family exists from $[r]$ to $[m]$, which means that $r \geq m$, so m/r is smaller than one so the inequality of the c -universality definition would hold for the above c' .

So what does this mean, and why is it helpful? Well, this is helpful because we could have a could simply use a 2-independent family to reduce a family that maps from $\mathbb{Z}_p$ to $\mathbb{Z}_p$ further down to $[m]$.

Orders of complexity

Jacob

Gonna cover some orders of complexity for convenience sake. Also, this will be an attempt at both formally and informally explaining the differences between big O, big Theta and big Omega.

$O(g(n))$ i.e. big O notation gives us an asymptotic **upper** bound on the complexity of what we're analyzing. So, if we have an $f(n) \in O(g(n))$ then that means that we can bound from above the run time of f by $g(n)$. Formally we would say:

Given a constant C and an n_0 , $0 \leq f(n) \leq Cg(n)$, $\forall n \geq n_0$.

So what does that mean? It means that the **order of complexity** is that given by $g(n)$, it doesn't have to be precisely $g(n)$, a constant can be modifying, but the order of magnitude is still given by $g(n)$. Note that this means that writing something like $O(2n)$ would make no sense, we would simply write $O(n)$. (This applies for big O, big Omega, and big Theta)

TL;DR: big O is an upper bound (asymptotically).

$\Omega(g(n))$, i.e. big Omega notation gives us an asymptotic **lower** bound on the complexity. If we have some $f(n) \in \Omega(g(n))$ then we can bound from below the complexity of f by $g(n)$. Much like with big O, we can express that formally as such:

Given a constant C and an n_0 , $0 \leq Cg(n) \leq f(n)$, $\forall n \geq n_0$.

Again, this means that the **order of magnitude** for the lower bound is defined by $g(n)$.

TL;DR: big Omega gives us a lower bound for the complexity.

$\Theta(g(n))$, i.e. big Theta gives us a tight bound in terms of $g(n)$. What does that mean? It means that we can express **both the lower and the upper bound** in terms of $g(n)$.

Formally we can express that by combining the above definitions:

Given constants C_1 , C_2 and an n_0 , $0 \leq C_1g(n) \leq f(n) \leq C_2g(n)$, $\forall n \geq n_0$.

This means we the order of magnitude for both the lower and the upper bound can be expressed in terms of $g(n)$ by multiplying by some constant. This is thought of as a tight bound because a single function can express both bounds.

TL;DR: big Theta means we can express both the upper and the lower bound in terms of one function.

Final note: the n_0 in the formal notations gives us a hint as to why these definitions are all asymptotic. Before this n_0 it doesn't really matter if $f(n)$ is larger or smaller than $g(n)$, we're interested in asymptotic behavior.

Questions

WARNING: THE FOLLOWING QUESTIONS ARE NOT COVERED:

Minor:

- Show how to parallelize an (a,b)-tree.
- Design and analyze a lock-free implementation of a stack.

Major:

- Formulate cache-aware and cache-oblivious algorithms for transposition of a square matrix. Analyze their time complexity and I/O complexity. - rather [in prerequisites](#)

[Minor] Describe a flexible array with growing and shrinking. Analyze its amortized complexity.

Kate, [lecture notes](#) + [lecture video](#) (starting from 1:01:38)

Intuition: When reserving space for an array that will store some numbers, we want to be efficient. To do this we implement growing and shrinking arrays. How do they work? Once we reach a certain threshold of occupied elements in the array, we either stretch the array or shrink it. Here we prove that this stretching and shrinking operation doesn't cost us much if we pick some proper thresholds and proper modifications to the capacity.

array:

capacity C , initially constant

size - n items which the array actually contains, initially 0

growing

usual append - constant time

when we hit the capacity - need reallocation which:

we create a new array of some higher capacity C' , copy all items to it, and delete the old array.

takes $\Theta(C')$

idea: double the capacity in each reallocation

amortized analysis: aggregation method - evaluate overall time for n appends

if we double the capacity on each reallocation, after k reallocations $C = 2^k$.

k reallocations take time $\Theta(2^1 + 2^2 + \dots + 2^k)$.

If we appended n items, we have k such that $2^{k-1} < n \leq 2^k$

2^{k-1} was not enough to contain n elements (that's why we did k -th reallocation)

but capacity of 2^k is enough for storing n elements (otherwise we would do $k+1$ -th reallocation).

and also $2^k < 2n$ (multiplied inequality above by 2).

$\Rightarrow n \leq 2^k < 2n$

$\Rightarrow 2^k \in \Theta(n)$ (by definition of Θ)

$\Rightarrow 2^1 + 2^2 + \dots + 2^k = 2^{k+1} - 2 \in \Theta(n)$

equality is just [some maths](#) (sum of geometric progression, simplified for this case)

the sum of geometric progression is asymptotically the last term of the progression, which means that the whole sum is $\Theta(n)$ [Video of the lecture 1, 1:10:21]

Jacob's explanation: you can see this clearly when you think of the binary number system. If you have e.g. 01111 (which is $2^0 + 2^1 + 2^2 + 2^3$), that really is just one away from 10000 which would be 2^4 .

So, n appends take $\Theta(n)$ time $\Rightarrow$ time of each append is constant.

shrinking

shrink when n is less than some part of capacity C .

if we choose $n < C / 2$:

if we have full array of capacity C (so that $n = C$), then the following sequence:

1. append 1 item $\Rightarrow$ increase capacity to $2C$, $n = C+1$
2. remove 1 item $\Rightarrow n = C$ (capacity is $2C$)
3. remove 1 more item $\Rightarrow n = C-1 < 2C/2 \Rightarrow$ decrease capacity from $2C$ to C
4. append 1 item $\Rightarrow n = C$, we're back in the initial state.

4 operations take $\Theta(C)$ time, which means that constant amortized time per operation is lost.

idea: shrink when $n < C / 4$

amortized analysis: accounting method - redistribute time between operations so that the total time remains the same, but the worst-case time decreases.

We have an arbitrary sequence of m operations (appends and removals).

We split this sequence to blocks, a block ends when the array is reallocated or the whole sequence of operation ends.

For each block, we analyze the cost of the reallocation at its end:

- The first block starts with capacity 1, so its reallocation takes constant time.
- The last block does not end with a relocation.
- All other blocks start with a reallocation, so at their beginning we have $n = C/2$.
 - If it ends with a stretch, n must have increased to C during the block.
 - at least $C/2$ pushes
 - If it ends with a shrink, n must have dropped to $C/4$.
 - at least $C/4$ pops
 - $\Rightarrow \Theta(C)$ operations in the block
 - reallocation takes $\Theta(C)$ time

$\Rightarrow$ we can redistribute the $\Theta(C)$ cost of the reallocation to $\Theta(C)$ operations in the block.

$\Rightarrow$ one operation gets only $\Theta(1)$ units of time.

Kirill [I could not understand the coolness of shrinking from this, so I'll try to explain it myself]: firstly, the intuitive explanation:

let's say that we have an array of capacity $C/2$ at the moment.

1. When do we need to stretch it? When it gets to the maximum of the $C/2$ elements
 - a. when we stretch it, we get $2 * C/2$ capacity, and the new array is half full
2. when to shrink? When it gets to half of $C/2$ elements (i.e. $\frac{1}{2} * C/2$).
 - a. when we shrink it, we get $\frac{1}{2} * C/2$ capacity, and the new array is half full

because in both situations the new array is only half full, in both cases there is “reserve” in both situations to make the pops and pushes in constant time. Which is cool.

This means that, whenever we make a shrink/stretch (whose cost is $O(C/2)$), we have gone through many pops/pushes, with cost of $O(1)$ each, but there were either $C/2$ of them (to stretch), or $C/4$ of them (to shrink) [so we can generalize $O(C/2)$ and $O(C/4)$ to $O(C/2)$]. In other words, each $O(C/2)$ stretch/shrink comes only after $O(C/2)$ pops/pushes, which means that they will be relatively “rare”, and they would depend on n the same way the pops/pushes depend on it. $\Rightarrow$ if complexity of n pushes is $O(n)$ and complexity of reallocations is “tightened” to them, we can divide $O(n)$ stretch/shrinks by n and get $O(1)$ complexity.

[Minor] Define binary search trees with lazy balancing. Analyze their amortized complexity.

Kate, [lecture notes](#) + [lecture video](#) (starting from 59:20)

Intuition: Perfectly balanced trees are nice and all and not hard to build but they are very expensive to maintain. Lazily balanced trees try to keep some of the nice properties while not costing as much to maintain. If we relax a bit “balancing” constraints we get quite good performance. Here we will analyze how relaxed things can get while keeping good performance.

Some notation:

For a node v of a binary tree:

- $T(v)$ denotes the subtree rooted at v (all descendants of v , including v itself).
- $s(v)$ is the size of v , i.e. the cardinality of $T(v)$, also called weight (!!).
- $l(v)$ and $r(v)$ is the left and right child of v .
 - If a child is missing, we have $l(v) = \emptyset$ and/or $r(v) = \emptyset$.
- $T(\emptyset) = \emptyset$ and $s(\emptyset) = 0$.

Just in case (this one is not a lazily balanced tree):

Definition: A tree is *perfectly balanced* if for each node v , we have $|s(l(v)) - s(r(v))| \leq 1$.

This means that the ratio of sizes of the children is close to 1:1.

Perfectly balanced tree is built from a sorted array at $O(n)$ time but is hard to maintain.

Lazily balanced tree means that we do not keep subtrees perfectly balanced and rebuild the structure only if there is a larger imbalance.

Definition: A node v is *in balance*, if for each its child c , we have $s(c) \leq \frac{2}{3} \cdot s(v)$. A tree is balanced, if all its nodes are in balance.

This means that the ratio of sizes of children is between 2:1 and 1:2.

This inequality is also called the invariant further.

Lemma: The height of a balanced tree with n nodes is $O(\log n)$.

Proof:

1. Let us follow an arbitrary path from the root to a leaf and keep track of node sizes.
2. The root has size n .
3. Each subsequent node has size at most $\frac{2}{3}$ of its parent's size (by definition of balanced tree).
 - a. the first child in the path has size: $\frac{2}{3} \cdot n$
 - b. the second child in the path has size: $\frac{2}{3} \cdot \frac{2}{3} \cdot n = (\frac{2}{3})^2 n$
 - c. etc. until we reach a leaf
4. At the end, we reach a leaf, which has size 1.

Zhenya's comment: By definition of a node in balance, when we are at the leaf, we get the inequality $1 \leq (\frac{2}{3})^x \cdot n$, where x is the number of nodes in the path

$$\Rightarrow (\frac{2}{3})^x \geq 1/n$$

$$\Rightarrow x \leq \log_{2/3}(1/n)$$

we change the sign when we apply log for base < 1 (Goutham & Jacobo, ty)

5. $\Rightarrow$ the path can contain at most $\log_{2/3}(1/n) = \log_{3/2} n = O(\log n)$ edges.

FIND operation

$O(\log n)$ worst-case complexity

because the height of the tree is $O(\log n)$.

INSERT operation

- Each node keeps track of its size.
- Every insertion adds a leaf to the tree so we have to add one to the sizes of all nodes between that leaf and the root.
- Along the path, we check that all nodes are still in balance.
- If balance is broken, we find the *highest* out-of-balance node v and rebuild the whole subtree rooted there to make it perfectly balanced. This takes $\Theta(s(v))$ time.

We can break only the path, other subtrees remain in balance.

Theorem: Amortized time complexity of the Insert operation is $O(\log n)$.

Proof:

Required prerequisite: [Potential](#)

Our potential should be like this:

- We would like to quantify, how far is the current tree from the perfectly balanced tree.
- Insertions should increase the potential gradually.
- When we need to rebalance a subtree, the potential should be large enough to pay for the rebalancing.

The potential will be defined as a sum of per-node contributions.

Each node will contribute the difference of sizes of its left and right child.

However, we must adjust the definition slightly to make sure that perfectly balanced trees have zero potential: if the difference is less than 2, we clamp the contribution to 0.

it is needed later so that the contribution of the node goes to 0 when we rebalance it.

$$\Phi := \sum_v \varphi(v), \quad \text{where}$$

$$\varphi(v) := \begin{cases} |s(\ell(v)) - s(r(v))| & \text{if at least 2,} \\ 0 & \text{otherwise.} \end{cases}$$

This potential is non-negative.

Initial potential is 0 (empty tree).

When we add a new leaf, the size of all nodes on the path to the root increases by 1.

The contributions of these vertices will therefore increase by at most 2 (they will usually change by exactly one, but because of the clamping, it can jump between 0 and 2).

0 if the subtree remains in perfect balance

2 in case if the perfectly balanced subtree goes out of its perfect balance (0 $\rightarrow$ 2)

[not mentioned in the lectures but I and Jacobo think that can happen] if insert makes the tree perfectly balanced (diff between children becomes 1), contribution to potential becomes 0 and the total potential decreases

If no rebuild took place:

we spent $O(\log n)$ time on the operation

height of the tree is $O(\log n)$

⇒ logarithmic way down, logarithmic adding of the leaf, logarithmic size increment along the path to the root

we increase the potential by $O(\log n)$

added a value from $[0, 2]$ to the potential for all nodes in the path

⇒ the total amortized cost is $O(\log n)$.

Rebuild:

Rebuild takes time $\Theta(s(v))$ but it happens infrequently enough for amortized cost of Insert to be logarithmic.

1. The invariant was broken for v and its child c .
Without loss of generality, c is the left child.
2. We have $s(l(v)) > \frac{2}{3} \cdot s(v)$
the invariant - $s(c) \leq \frac{2}{3} \cdot s(v)$ and here it's broken.
3. so the other subtree must be small: $s(r(v)) < \frac{1}{3} \cdot s(v)$.
because $s(c) = s(l(v)) + s(r(v))$ and the left child is $> \frac{2}{3} \cdot s(v)$
4. ⇒ The contribution of v is $\phi(v) > \frac{1}{3} \cdot s(v)$.
 $\phi(v) = |s(l(v)) - s(r(v))|$ so that if one child is more than $\frac{2}{3}$ of v and the other is less than $\frac{1}{3}$ of v , then the difference will be more than $\frac{1}{3}$ of v .
5. After the rebuild, this contribution becomes zero and contributions of all other nodes in the subtree also become zero.
because rebuild is making the subtree perfectly balanced and perfectly balanced subtree has potential of 0.
6. All other contributions stay the same.
we do not touch them, subtrees other than our path are not affected by Insert
7. ⇒ The whole potential decreases by at least $\frac{1}{3} \cdot s(v)$.
this was the previous contribution of the node before rebalancing, so this contribution is "released" by rebalancing (after rebalancing the contribution is 0), and it can pay for the rebuild.
8. The real cost of the rebuild is $\Theta(s(v))$.
9. ⇒ we can multiply the potential by a suitable constant, this change in potential can cover the real cost.
10. ⇒ zero amortized cost of the rebuild.

Usual inserts without rebuilds increase the potential and rebuild spends it.

Overall, this means that the amortized time complexity of the Insert operation is $O(\log n)$.

DELETE operation

TODO: this was left as an exercise

From [practical 2 solutions](#) (not sure this is needed for the exam):

Consider the three possibilities: a) deleted node is a leaf, b) deleted node has just one child, c) deleted node has two children. In all three cases, you need $O(\log(n))$ to perform the operation. The size of all nodes on the path to the root decreases by 1. The contributions of these vertices will therefore increase by at most 2 (they will usually change by exactly one, but because of the clamping, it can jump between 0 and 2). So we increase the potential at most by another $O(\log(n))$, so the total amortized cost is $O(\log(n))$. When rebuilding a subtree rooted by v , its potential is $\Phi(v) > \frac{1}{3} \cdot s(v)$, which is enough for building the balanced subtree (in linear time).

[Major] Define the Splay tree. State and prove the theorem on amortized complexity of the Splay operation.

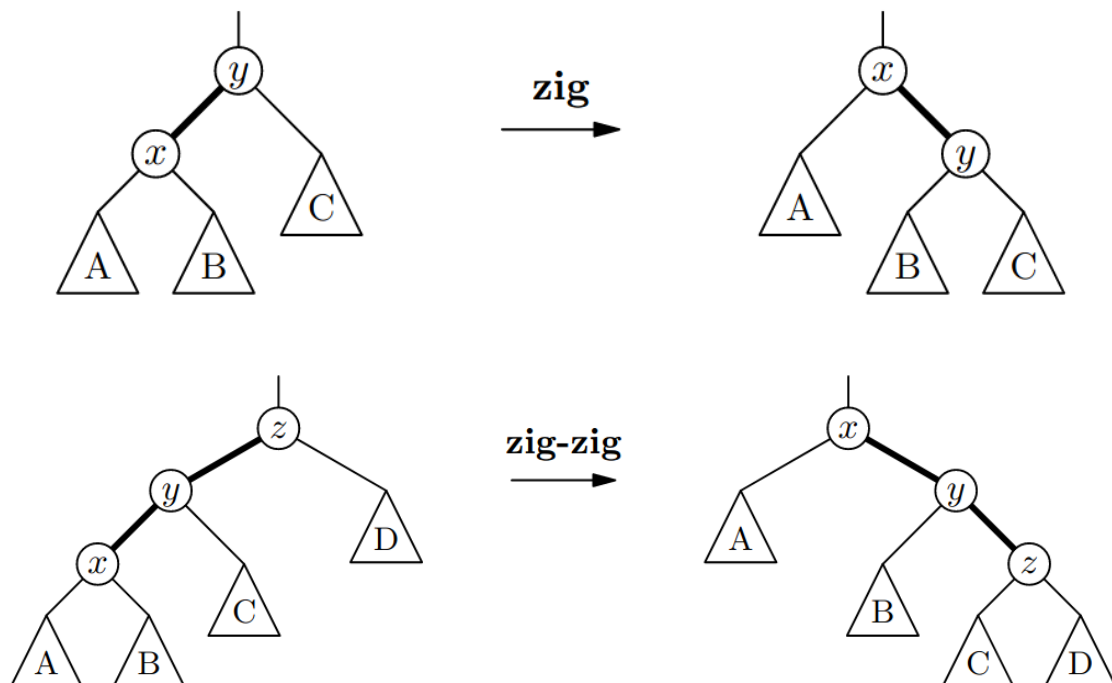
Jacob, last year's and this year's notes

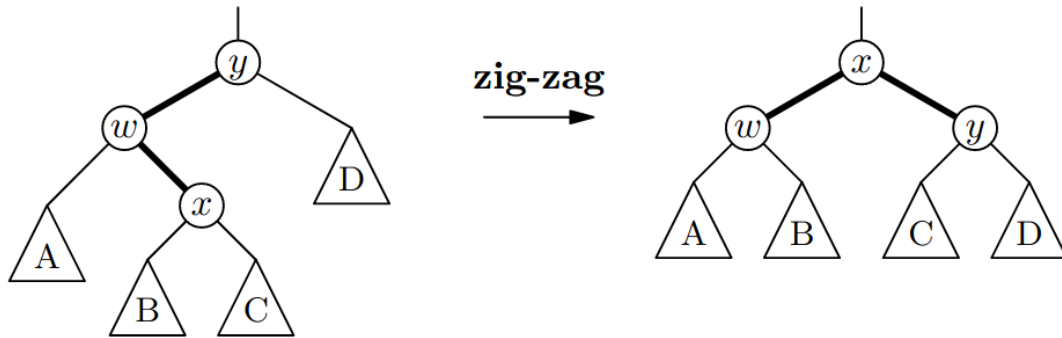
Intuitive explanation: Splay trees are a type of self-balancing trees. What that means is that every time we access, insert or delete an element from the tree, it does a set of operations that adjusts its structure such that it remains close to being a balanced tree. In the case of splay trees it does the so-called splay operation. We do this operation as opposed to naive rotations of the tree because it is a way more efficient to keep the tree balanced, i.e. we move around less nodes on average, especially given some sequences of inserts and deletes. The splay operation leads to a nice [amortized] complexity of the insert, find and delete operations.

Prereq: the [potential](#) concept

Definition: A splay tree is a self-adjusting binary tree. Unlike a vanilla BST, in a splay tree, after accessing a node, we rotate it to the root using a series of rotation operations. These operations over the node to the root are called splay operations. In order to complete this definition we need to define the operation $\text{SPLAY}(x)$:

A splay operation consists of a series of *zig*, *zig-zig* and *zig-zag* operations. These are defined in the following image (in all of them we are rotating the x node to the root of the shown subtree).





As we can see, the idea here is that we do double rotations whenever possible.

Theorem: The amortized cost of $\text{Splay}(x)$ is at most $3(r'(x) - r(x)) + 1$ which is $O(\log n)$, where $r(x)$ is the rank of the node x before the operation and $r'(x)$ the rank after.

Note: First we need to realize that it is defined this way because we will analyze the operation step by step. Otherwise it would make no sense to consider the rank of the final operation since x will always end up at the root. This is not really too important but it's easier for me to understand the definition of the theorem if I take this into account. Also, this fits nicely with our first encounter with potential analysis.

First, we introduce some notation, without it we can't even understand the theorem:

- $T(v)$ is the subtree rooted at node v
- $s(v)$ is the cardinality of $T(v)$. Note that: $0 \leq s(v) \leq n$
- $r(v) = \log s(v)$. Note that: $0 \leq r(v) \leq \log n$
- n is the total number of nodes in the tree
- Φ (potential) of the tree is $\sum_v r(v)$. Note that: $0 \leq \Phi \leq n \log n$

Note: it follows from the previous definitions that an empty tree starts out with a potential of 0 hence the lower bound in the inequalities. In the context of the proof, $r(x)$ is the rank of node x before an operation and $r'(x)$ is the rank after the operation was performed.

So now we can see what the theorem is actually saying and why it equates to $O(\log n)$.

Proof layout:

1. Understand that the amortized cost of each operation (zig-zig, zig-zag and zig) can be bounded by simple expressions.
2. Understand that the total cost is the sum of the cost of all operations.
3. Use the concept of a [telescopic sum](#). (This is basically just a series in which terms cancel each other out and thus determining the total is trivial in the end.)

Proof:

We first claim (and we will then prove) that:

1. Amortized cost of $\text{zigzag}(x) \leq 3(r'(x) - r(x))$
2. Amortized cost of $\text{zigzig}(x) \leq 3(r'(x) - r(x))$
3. Amortized cost of $\text{zig}(x) \leq 3(r'(x) - r(x)) + 1$

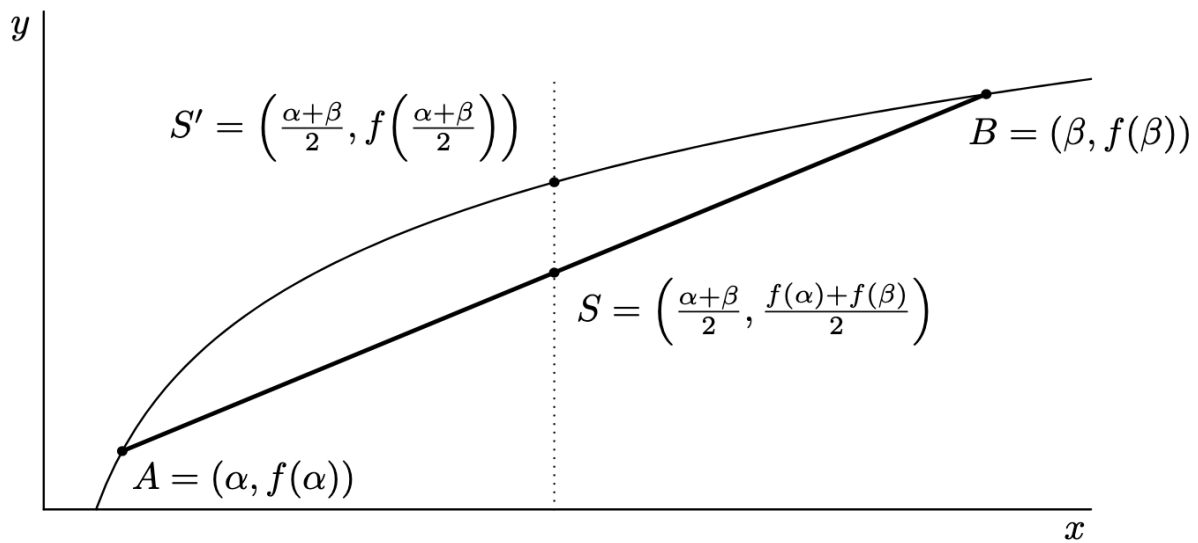
Now, to prove these claims we need to take a brief look at how logs work.

We really only need to take advantage of the concavity of logarithms. Using a special case of Jensen's inequality we can see that:

$$\forall \alpha, \beta > 0, \log \frac{\alpha+\beta}{2} \geq \frac{\log \alpha + \log \beta}{2}$$

$$\text{We then can also have that: } \log \alpha + \log \beta \leq 2(\log(\alpha + \beta)) - 2 \quad (1)_{cg}$$

What this is basically saying is that the log of the average of two values is greater than the average of the logs. The concavity basically makes it so the curve goes above the line that joins together the values of the logs. This is illustrated here:



Now we're ready to prove the above claims.

Note: Please reference the images of the subtrees that we used above when defining the different operations to know what w , x , y , z mean in the following. Also, remember that $r(x)$ and $r'(x)$ are the ranks of node x before and after the operation respectively.

$$1. \text{ Amortized cost of } \text{zigzag}(x) \leq 3(r'(x) - r(x))$$

First we define the amortized cost for $\text{zigzag}(x)$ as (and we try to bound each rank by $r(x)$ and $r'(x)$):

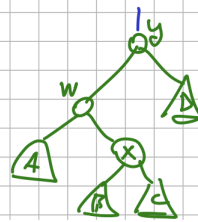
Reminder: $A = R + \Delta\Phi$ and $R = 2$ here (real cost is the number of rotations during operation)

$$A = 2 + r'(w) + r'(x) + r'(y) - r(w) - r(x) - r(y) \text{ basing the 2 off the real cost}$$

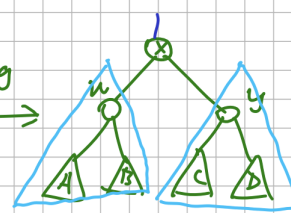
$$r'(w) + r'(y) \leq 2r'(x) - 2 \text{ by equation (1) (Converting to logs and then bounding by (1))}$$

$$r(w) + r(y) \geq 2r(x) \text{ due to the fact that } T(x) \subseteq T(w), T(y)$$

We finally get that $A \leq 3(r'(x) - r(x))$ (since we put everything in terms of x).

Case 1: zig-zag

zig-zag



$$\begin{aligned} T(w) &\geq T(x) \\ s(w) &\geq s(x) \\ r(w) &\geq r(x) \\ -r(w) &\leq -r(x) \end{aligned}$$

real cost = 2

$$\text{am. cost} = 2 + \Delta\Phi = 2 + \underbrace{r'(w)}_{\leq -r(x)} + r'(x) + \underbrace{r'(y)}_{\leq -r(x)} - \underbrace{r(w)}_{\leq -r(x)} - \underbrace{r(x)}_{\leq -r(x)} - \underbrace{r(y)}_{\leq -r(x)}$$

$$\begin{aligned} &\log s'(w) + \log s'(y) \\ \text{Lemma 5} &\leq 2 \log (s'(w) + s'(y)) - 2 \\ &\leq s'(x) \\ &\leq 2r'(x) - 2 \end{aligned}$$

$$\leq 3r'(x) - 3r(x)$$

2. Amortized cost of $\text{zigzig}(x) \leq 3(r'(x) - r(x))$

First we define the amortized cost for $\text{zigzig}(x)$ as (and then we try to bound each rank by $r(x)$ and $r'(x)$):

$A = 2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z)$ basing the 2 off the real cost

$r(x) + r'(z) \leq 2r'(x) - 2$ by equation (1) (Converting to logs and then bounding by (1))

we join $r(x) + r'(z)$ because these subtrees contain almost all nodes except for y

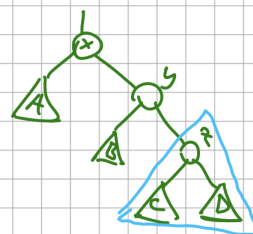
$$\Rightarrow r'(z) \leq 2r'(x) - r(x) - 2$$

$r'(y) \leq r'(x)$; $r(y) \geq r(x)$; $r(z) = r'(x)$ you can deduce these by looking at the diagrams

Through this we get that $A \leq 3(r'(x) - r(x))$

Case 2: zig-zig

zig-zig



real cost = 2

$$\text{am. cost} = 2 + \Delta\Phi = 2 + r'(x) + \underbrace{r'(y)}_{\leq r'(x)} + \underbrace{r'(z)}_{\leq 2r'(x) - r(x) - 2} - \underbrace{r(x)}_{\leq -r(x)} - \underbrace{r(y)}_{\leq -r(x)} - \underbrace{r(z)}_{\leq -r(x)}$$

Use Lemma 5:

$$\begin{aligned} r(x) + r'(z) &= \log s(x) + \log s'(z) \\ &\leq 2 \log (s(x) + s'(z)) - 2 \\ &\leq s'(x) \end{aligned}$$

$$\begin{aligned} &\leq 2r'(x) - 2 \\ \hookrightarrow r'(z) &\leq 2r'(x) - r(x) - 2 \end{aligned}$$

$$\leq 3r'(x) - 3r(x)$$

3. Amortized cost of $\text{zig}(x) \leq 3(r'(x) - r(x)) + 1$

First we define the amortized cost of $zig(x)$ as the following (and then we try to bound each rank by $r(x)$ and $r'(x)$):

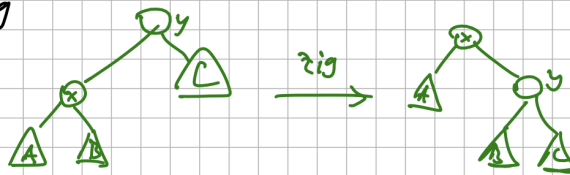
$A = 1 + r'(x) + r'(y) - r(x) - r(y)$ basing the 1 off the real cost

$r'(x) \geq r'(y)$; $r(x) \leq r(y)$; you can deduce these by looking at the diagrams

$A \leq 2(r'(x) - r(x)) + 1 \leq 3(r'(x) - r(x)) + 1$ we simply extend it to 3 so we can use it in the final sum.

Now that we have proven all three claims, we can now use all of these as the bounded amortized costs and sum them all up to get the final amortized cost for the Splay(x) operation.

Case 3: zig



real cost = 1

$$\text{am. cost} = 1 + \Delta\Phi = 1 + r'(x) + \overset{\leq r'(x)}{r'(y)} - \overset{\leq -r(x)}{r(x)} - r(y)$$

$$\leq 1 + 2r'(x) - 2r(x) \leq 1 + 3r'(x) - 3r(x)$$

because $r'(x) - r(x) \geq 0$

But if we sum all of these at each step:

$$A \leq \sum_{i=1}^t 3(r'_i(x) - r_{i-1}(x)) + 1$$

But this is a simple telescopic series. If we expand it we can clearly see that each term is there twice, once positive and once negative *except* for the terms $r'_0(x)$ and $r_t(x)$. This means that the sum is equal to:

$$3(r'_t(x) - r_0(x)) + 1$$

Thus proving our theorem.

[Minor] Design operations Find, Insert, and Delete on a Splay tree. Analyze their amortized complexity.

Jacob, last year's and this year's notes

Prereq: [potential](#), [splay trees](#)

Intuitive explanation: this basically boils down to splaying the lowest visited node. This proves to be better at balancing the trees and keeping the amortized complexity fast compared to naive rotations. This applies for *FIND*, *INSERT* and *DELETE*, which, turns out, all have $O(\log n)$ time complexity.

FIND(x): we do a regular *search(x)* and then splay over the lowest visited node. This node will be *x* if the search was successful. What is the complexity of this?

Well, let's assume that node *x* is at depth *d*. Going from the root to the node has real cost $\Theta(d)$ and we do not change the potential of the tree when going down. Then we need to splay, which has real cost $\Theta(d)$, *however* it amortizes to $O(\log n)$ as per the theorem proved in the previous topic. This means that we can simply absorb the cost of going down by the amortized complexity of splaying. Thus, *FIND* has $O(\log n)$ time complexity.

INSERT(x): now we define our insert. Similarly, we simply run a search for *x* and if it is found we do not insert anything, and we splay the last node. This is equivalent to doing *FIND(x)*. However, if *x* is not found, we insert it as a leaf where it corresponds and then splay it. But what about the potential change of adding a new leaf? Adding it has constant real cost, but the potential of the tree definitely changes.

NOTE: we use the same notation as in the previous question.

Lemma: Adding a leaf to the tree increases potential by $O(\log n)$.

Proof: for this proof we define the path traversed for reaching the node where we will insert as $v_1, v_2, \dots, v_t$ and the inserted node will be v_{t+1} .

For this we get that $\Delta\Phi = r'(v_{t+1}) + \sum_{i=1}^t (r'(v_i) - r(v_i))$.

Let's analyze this by parts:

- $r'(v_{t+1})$ is zero because we know it's a new node and thus it's the lowest rank possible.
- By definition, we know that $r'(v_i)$ is the $\log(s(v_i)) + 1$ since we added the leaf. BUT! We know that this is bounded from above by the rank of its parent node. So we have that $r'(v_i) \leq r(v_{i-1})$. Why is this helpful? Well, we can bound the sum from above by telescoping like in the previous question:
 $\Delta\Phi \leq r'(v_1) - r(v_t)$ and we know that the ranks are logarithmic so we know that the subtraction is $O(\log n)$.

By this we have shown that the change in potential is bounded by $O(\log n)$. Real cost is constant, potential difference is bounded by $O(\log n)$, and $A = R + \Delta\Phi$. Therefore, we can definitely conclude that the *INSERT* operation can be done in time complexity $O(\log n)$.

DELETE(x): Here we basically can do the same analysis as with the *FIND* operation. So, we need to traverse the tree to find x , if it is present, then we have 3 possible cases:

1. x is a leaf
2. x has one child
3. x has two children

The first two cases are trivial, as we simply splay over the parent of the removed node. In the case the node has a child, then we replace the removed node with the child. We then splay the parent of the removed node. As we have shown before, this has amortized complexity of $O(\log n)$. If the node has two children, we need to pick either the max of the left subtree or the min of the right subtree to substitute the deleted node. And then we splay the parent of the original node.

What about the change in potential for removing the node? Well, in this case we are removing a node so the potential cannot go up. In this case the change in potential is in our favor and we do not need to worry like with *INSERT*.

Thus we conclude that *DELETE* has time complexity of $O(\log n)$.

All operations have $O(\log n)$ amortized time complexity.

[Major] Define the (a,b)-tree. Describe operations Find, Insert, and Delete. Analyze their complexity in the worst case.

Zhenya

Definition: A **multi-way search tree** is a rooted tree with specified order of children in every node. Nodes are divided to internal and external.

Each **internal** node contains one or more distinct keys, stored in increasing order. A node with keys $x_1 < \dots < x_k$ has $k+1$ children $s_0, \dots, s_k$. The keys in the node separate keys

in the corresponding subtrees. If we extend the keys by indicators $x_0 = -\infty$ and

$x_{k+1} = +\infty$, every key y in the subtree $T(s_i)$ satisfies $x_i < y < x_{i+1}$.

External nodes carry no data and they have no children. These are the leaves of the tree. In a program, we can represent them by null pointers.

Definition: An **(a,b)-tree** for parameters $a \geq 2$ and $b \geq 2a - 1$ is a multi-way search tree, which satisfies:

1. The root has between 2 and b children (unless it is a leaf, which happens when the set of keys is empty). Every other internal node has between a and b children. [Subsequently, for internal node $a-1 \leq \# \text{ keys} \leq b-1$]
2. All external nodes have the same depth i.e they are on the same level.

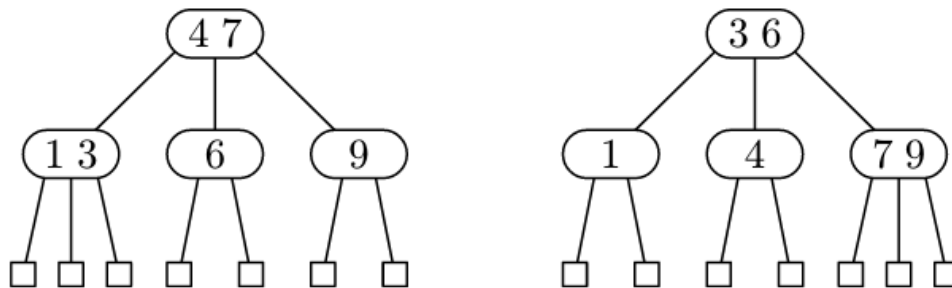


Figure 3.1: Two (2, 3)-trees for the same set of keys

Lemma: The height of an (a, b)-tree with n keys lies between $\log_b(n + 1)$ and $1 + \log_a(\frac{n+1}{2})$.

Proof:

1. Let us start with the upper bound. Upper bound for the height of the tree with n keys is the case when each node stores the fewest keys possible (i.e. $a - 1$), so that the tree “stretches vertically” to fit all n keys.

We will calculate the minimum number of keys in a tree of height h . The minimum will be attained by a tree where each node contains the minimum possible number of keys, so it has the minimum possible number of children. Level 0 contains only the root with 1 key. Level h contains only external nodes. The i -th level in between contains $2a^{i-1}$ nodes with $a - 1$ keys each. [Kate] It's 1 node (root) at level 0, 2

nodes at level 1 (because root has 1 key) and a nodes at all subsequent levels, so at the i -th level it's $1 \cdot 2 \cdot a \cdot \dots \cdot a = 2a^{i-1}$.

Summing over levels, we get:

$$1 + \sum_{i=1}^{h-1} 2a^{i-1}(a-1) = 1 + 2(a-1) \sum_{i=0}^{h-2} a^i = 1 + 2(a-1) \frac{(a^{h-1} - 1)}{(a-1)} = 2a^{h-1} - 1.$$

So $n \geq 2a^{h-1} - 1$ in any tree of height h . Solving this for h yields $h \leq 1 + \log_a \left(\frac{n+1}{2}\right)$.

- For the lower bound, we consider the maximum number of keys for height h . All nodes will contain the highest possible number of keys. Thus we will have b^i nodes at level i , each with $b-1$ keys. In total, the number of keys will reach

$$\sum_{i=0}^{h-1} b^i(b-1) = (b-1) \sum_{i=0}^{h-1} b^i = (b-1) \cdot \frac{b^h - 1}{b-1} = b^h - 1.$$

Therefore in each tree, we have $n \leq b^h - 1$, so $h \geq \log_b(n+1)$.

Corollary: The height is $\Omega(\log_b n)$ and $O(\log_a n)$.

FIND operation

Reminder: height of tree is $O(\log_a n)$ and the number of keys in the node is at most $b-1$.

Time complexity is $O\left(\frac{\log b}{\log a} \log n\right)$ because it visits $O(\log_a n)$ nodes, which is $O\left(\frac{\log n}{\log a}\right)$. In each node, it compares x with all keys of the node, which can be performed in time $O(\log b)$ by binary search.

INSERT operation

- We are looking for a key we want to insert
- If the key is not present yet, the search ends in a leaf (external node). We cannot simply turn this leaf into an internal node with two external children — this would break the axiom that all leaves lie on the same level.
- We insert the key to the parent of the external node — that is, to a node on the lowest internal level. Adding a key requires adding a child, so we add a leaf.
- If the node has more than $b-1$ keys, we need to split the overfull node: the middle key is moved to the parent, all smaller keys form one new node, and all larger keys form the other one.
- If we need to split the root, the height of a tree will increase by 1.

Time complexity of Insert is $O\left(b \frac{\log n}{\log a}\right)$. [In the lecture notes it is given with Θ , but in the lecture it was O] In the worst case, we have to split all way to the root, which takes $O\left(\frac{\log n}{\log a}\right)$ and we have to rearrange all keys which will take $O(b)$ time.

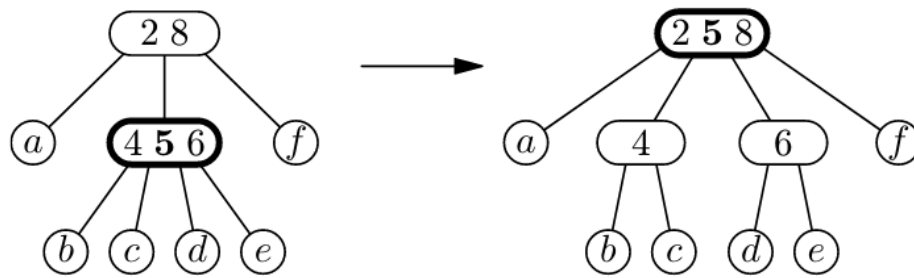
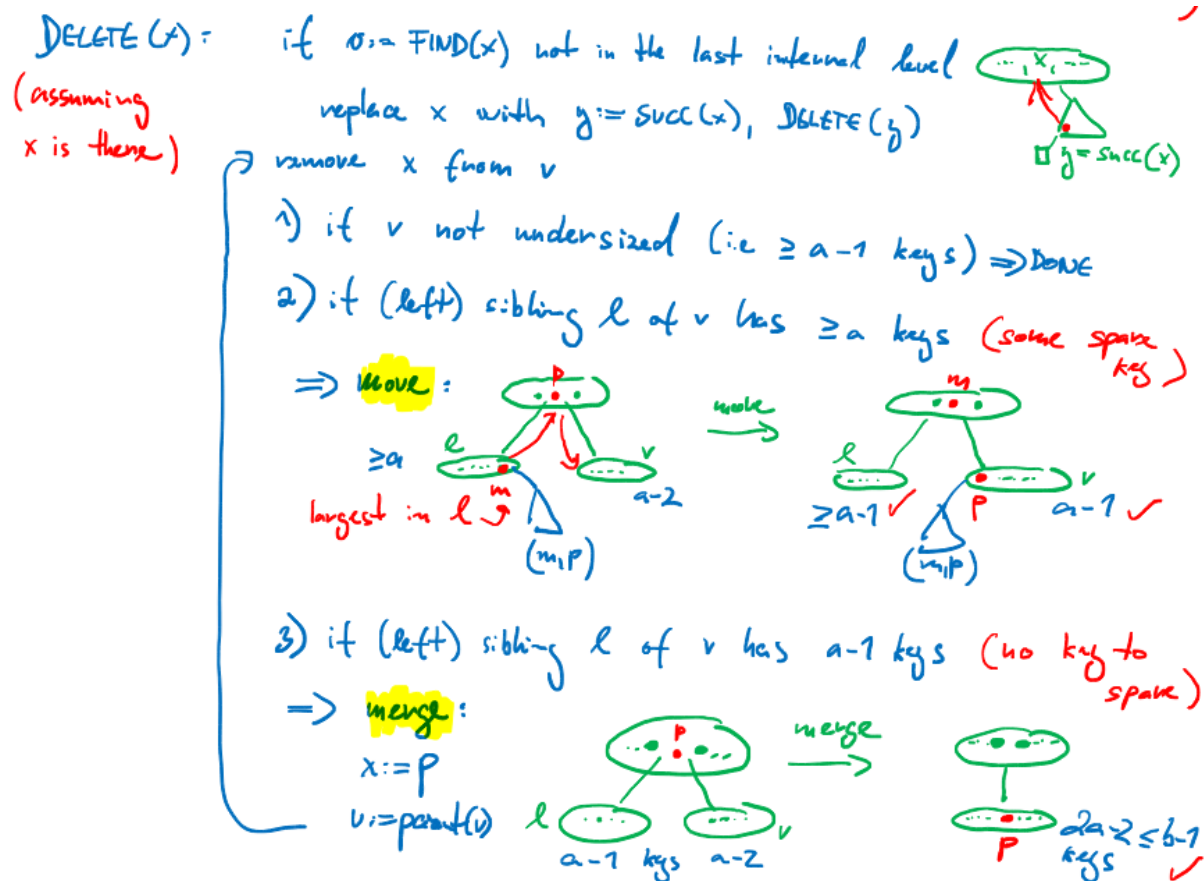


Figure 3.2: Splitting an overfull node on insertion to a $(2, 3)$ -tree

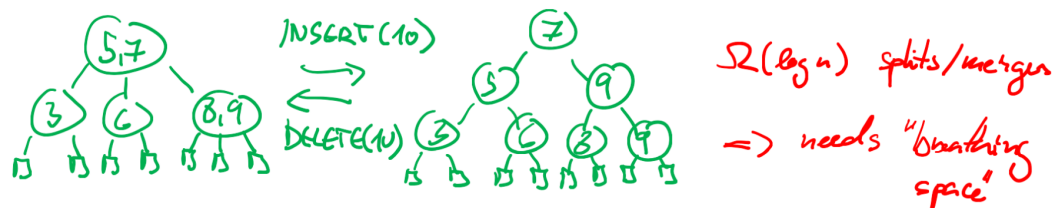
DELETE operation

- We have to find the key first. If it is located on the last internal level, we can delete it directly, together with one of the leaves under it.
- If the key is located on the higher level, we replace the deleted key by its successor (which is the leftmost key in the deleted key's right subtree). The successor lies on the last internal level, so it can be deleted directly.
- Then we have to fix possibly undersized nodes.



Time complexity of Delete is $O(b \frac{\log n}{\log a})$. Again in the worst case we have to go through all levels which would take $O(\frac{\log n}{\log a})$ and we have to rearrange all keys which will take $O(b)$ time.

☹️ $(a, 2a-1)$ -tree cannot have $O(1)$ amortized modification cost for mixed INSERTs and DELETEs.



Theorem: A sequence of m Inserts and Deletes on an initially empty $(a, 2a)$ -tree performs $O(m)$ node modifications.

Proof: (required prerequisites [Potential](#))

Potential will be a sum of contributions of every node.

$$\Phi = \sum_{\text{all nodes } v} f(\# \text{keys in } v)$$

$f: N \rightarrow N$

The number of keys will be in the range of $a - 2$ to $2a$, because there will be underfull and overfull nodes.

Observation: initially $\Phi = 0$ and potential should be always be non-negative, $\Phi \geq 0$.

Idea: we would like to have amortized cost 0 for split/merge operations. The real cost would be 1 (it's actually 3 but we can make it 1 by scaling the cost and the potential).

Requirements on the function f :

1. $|f(i) - f(i + 1)| \leq c$ for some constant c . i is some number of keys in the node.

This means that if the number of keys in the node changes by 1, the node's contribution changes at most by a constant.

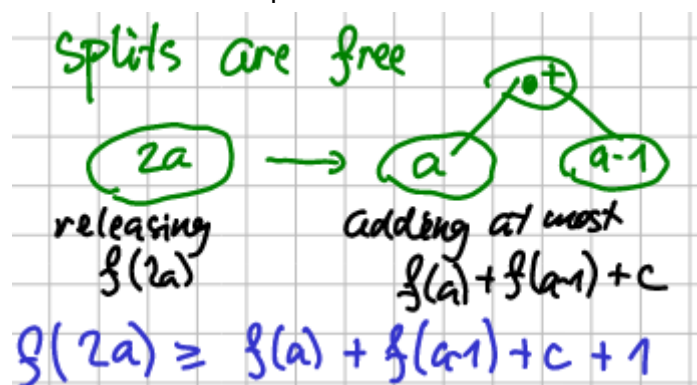
2. $f(2a) \geq f(a) + f(a - 1) + c + 1$

Idea is to have free splits.

The released potential is $f(2a)$, which corresponds to a node having too many keys.

It goes to

- $f(a) + f(a - 1)$, two new nodes created in the result of split,
- c which is a cost of adding a key to a parent node of newly created nodes,
- 1 is the real cost of the operation



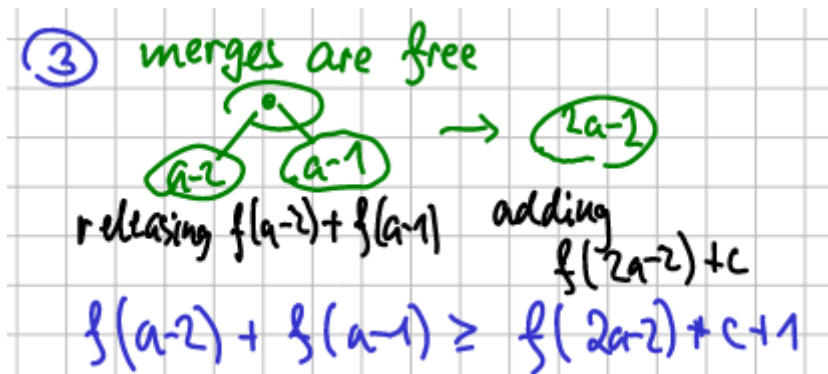
3. $f(a - 2) + f(a - 1) \geq f(2a - 2) + c + 1$

Idea is to have free merges.

The released potential is $f(a - 2) + f(a - 1)$ corresponding to one underfull node and one node with the minimum number of keys.

The right part is:

- $f(2a - 2)$ for a new resulting node after merge
- c for removing a key from parent
- 1 is the real cost of the operation



We have to find such a function f . The function can be the following with $c = 2$:

k	$a - 2$	$a - 1$	a	...	$2a - 2$	$2a - 1$	$2a$
$f(k)$	2	1	0	...	0	2	4

The function is zero between a and $2a - 2$.

The function f satisfies all three requirements.

Amortized cost of Insert equals $O(c)$ [initial insertion, req 1 above] + 0 [per split].

Amortized cost of Delete equals $O(c)$ [initial delete] + 0 [per merge] + $O(c)$ [stealing key]

So, the amortized cost of both operations is constant. As the potential is non-negative and initially zero, this guarantees that the total real cost of all operations is $O(m)$.

[Minor] Analyze k-way Mergesort in the cache-aware model. Which is the optimum value of k?

Kate, [lecture notes](#) + [lecture video](#) (starting from beginning)

Intuition: In this context we want to also worry about I/O performance and not just time complexity. Here we will analyze what happens when we merge k blocks at the same time as opposed to just two. We will see that this does not impact our good time complexity and that picking a fitting k will lead to good I/O performance.

Prerequisites:

[External memory model aka I/O model](#)

and [Cache models \(cache-aware and cache-oblivious\)](#)

Main concept of Mergesort: sort two parts of array and merge them into one sorted part.

Approach:

array of size N

On each step, we have the array divided into runs - sorted ranges of array

K is a number of simultaneous runs that are merged into one at one pass

it's kind of a hyperparameter

All runs except for the last one have the same size

We start with 1-item runs

In each pass, we merge K runs into one

remark: we just copy all remaining runs which didn't "fit" into the merge

⇒ Resulting runs are K times bigger.

⇒ After the i^{th} pass, we have K^i -item runs, so we stop after $\lceil \log_K N \rceil = \lceil \log N / \log K \rceil$ passes.

One merge:

we merge T items

basic idea: when merging K runs, have pointers in each of these K runs

one step:

find the minimum of values at this pointers - using heap

usual min is linear in K , ExtractMin in heap - $\Theta(\log K)$

add the minimum to the output

move the corresponding pointer.

Insert in heap - $\Theta(\log K)$

⇒ $\Theta(\log K)$ for one step (because of heap)

⇒ merging T items takes $\Theta(T \log K)$ time

⇒ the whole Mergesort takes $\Theta(N \log K \cdot \log N / \log K) = \underline{\underline{\Theta(N \log N)}}$ for any K

one pass - $\Theta(N \log K)$ and we have $\lceil \log N / \log K \rceil$ passes

I/O complexity:

If cache is large enough, number of reads at one merge - $O(T / B + 1)$

total number of all scanned items is $2T$ (inputs + outputs), we omit 2 in asymptotics

B is the size of a cache block

we read block-wise

Here, we could've had $T / B + K$ because in case our runs were scattered in the memory, we would have to "round up" every such standalone block. But our runs are consecutive (follow each other in the memory) so we can only add 1 and be fine. heap (for finding the minimum when merging) will always be stored in cache

⇒ no I/O operations for it.

⇒ the whole Mergesort: $O(N / B \cdot \log N / \log K + 1)$

one pass: $O(N / B + 1)$ (+1 because all runs are consecutive)

+1 in final formula for all steps:

if this +1 for one step is significant, then all runs fit into the memory and all operations are made in cache, so adding just 1 is sufficient to cover this

Optimal K:

Size of cache

M is the size of cache.

For merging, we need to store in cache:

K blocks of runs

1 block of output

K values of heap

some variables (constant)

⇒ we need $M \geq (K+1) \cdot B + K + O(1)$ values

⇒ $M \geq 2KB$ is sufficient.

Goal: set K as large as possible.

$K = M / 2B$ (from previous inequality) which is asymptotically the size of the block.

⇒ the whole Mergesort: $O(N / B \cdot \log N / \log (M / B) + 1)$

⇒ More efficient if cache is larger

Overall:

Time complexity: $\Theta(N \log N)$ for any K

I/O complexity:

$O(N / B \cdot \log N / \log K + 1)$

Optimal K: $K = M / 2B$

if K is optimal, then $O(N / B \cdot \log N / \log (M / B) + 1)$

[Major] Formulate a cache-oblivious algorithm for transposition of a square matrix. Analyze its time complexity and I/O complexity.

Kate, [lecture notes](#) + [lecture video](#) (starting from 1:13:00) + [this year's notes](#)

Intuition: We will design an algorithm that transposes a square matrix and which doesn't need to know what the parameters of the cache are in order to perform well. We will do this using a clever divide and conquer approach that is designed in such a way that once we reach a level of recursion, caching will be ideal.

Prerequisites:

[External memory model aka I/O model](#)

and [Cache models \(cache-aware and cache-oblivious\)](#)

Input: square matrix of size $N \times N$.

Task: transpose the matrix in a way which will be effective in terms of I/O complexity.

Cache-oblivious algorithm

Idea: we can design an optimal algorithm for cache-aware model using tiles, but this algorithm depends on the parameters of the cache (tile size $d = B$). Here we will use the divide and conquer approach, which processes matrix quadrants recursively.

1. Split the matrix into quadrants - tiles of $N/2$ size.

A_{11}	A_{12}
A_{21}	A_{22}

2. We will use recursion to transpose the matrix.

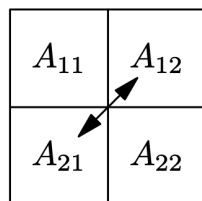
On each step, divide the current [sub]matrix to quadrants.

A_{11} and A_{22} can be just transposed recursively.

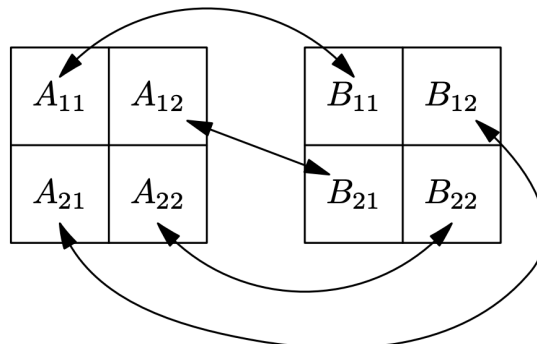
for A_{12} and A_{21} , we need to transpose and swap them.

But transposing and then swapping is expensive in time - $O(N^2 \log N)$

More effective way:



Transpose (T) for one matrix:
 A_{11} and A_{22} are again Transpose (T),
 A_{21} + A_{12} are one Transpose and Swap (TS)



Transpose and Swap (TS) for two submatrices:
 4 Transpose and Swap (TS):
 two diagonals ($A_{11} + B_{11}$, $A_{22} + B_{22}$),
 2 off-diagonals ($A_{21} + B_{12}$, $A_{12} + B_{21}$)

3. Recursion stops when we reach 1×1 "matrices", i.e. individual items

More explicit explanation of Transpose (T) and Transpose and Swap (TS) operations:

T takes one input matrix, TS takes two input matrices.
 In our initial matrix, we use 2 T for quadrants A_{11} and A_{22} .
 recursive T:

- split input [sub]matrix into quadrants
- 2 T on diagonal tiles
- 1 TS on off-diagonal tiles

In our initial matrix, we use 1 TS for A_{21} and A_{12} (in the image, they are A and B).
 Recursive TS:

- split two input [sub]matrices A and B into quadrants
- 4 TS for corresponding (see image) tiles between A and B
- example of the whole process:

close to the end of recursion, there will be TS between the left bottom corner of A_{21} and right top corner of B_{12} (try to imagine multiple recursive splits into quadrants and TS between them). that's how we manage to transpose the whole matrix

$\Rightarrow$ T recurses to 3 subproblems, TS recurses to 4 subproblems.
 All these sub-problems have size $N / 2$.

Time complexity

Tree of recursion:

every node has 3 or 4 children (T $\rightarrow$ 3 subproblems, TS $\rightarrow$ 4 subproblems)

at level i , we have at most 4^i nodes with subproblems of size $N / 2^i$

root - level 0

nodes at level 2: 3 (from first T) + 3 (from second T) + 4 (from TS) = 10 $<$ 4^2

size of subproblem at level 2: $N / 2 / 2 = N / 2^2$

$\Rightarrow$ the height of the tree is $\log N$

$\Rightarrow$ tree has at most $4^{\log N} = N^2$ leaves.

every node has more than 2 children $\Rightarrow$ there are less internal nodes than leaves

Leaves - 1x1 "matrices", individual items.

- In every TS leaf, we swap a pair of items.
- In every T leaf, nothing happens
- input to T leaf is just 1 item from the main diagonal
- Internal nodes only redistribute work and they do not touch items.

$\Rightarrow$ every node takes $O(1)$ time

$\Rightarrow$ the whole algorithm finishes in $O(N^2)$ steps

this recursive algorithm accesses every item exactly once and there are N^2 items

I/O complexity

First, we need to turn to the cache-aware model to find the optimal size of the tile.

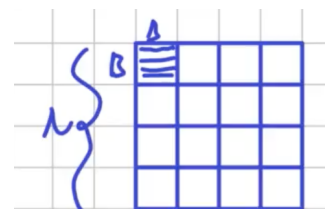
Suppose we know B and M (parameters of the cache).

We divide the matrix into tiles of size d .

number of tiles: $\lceil N / d \rceil^2 \leq (N / d + 1)^2 \in O(N^2 / d^2 + 1)$.

this +1 here again for the case if d is asymptotically larger than N

Best size of tile: $d = B$. Proof:



if N is a multiple of B - a tile will be a set of blocks.

⇒ each row of a tile is a block

⇒ a tile includes B full blocks

⇒ we can process a tile in $O(B)$ I/O operations.

reads to load tile into cache = B

writes to write the tile back to matrix = B

we have $N^2 / d^2 = N^2 / B^2$ tiles

⇒ the total I/O complexity is $O(N^2 / B)$.

if N is not a multiple of B - a tile will include some block boundaries

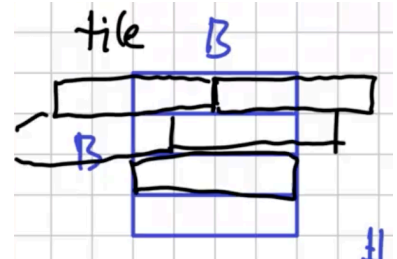
each row spans max 2 blocks

⇒ we can process a tile in $2B = O(B)$ I/O operations

we have $\lceil N / B \rceil^2 = O(N^2 / B^2 + 1)$ tiles

⇒ the total I/O complexity is $O(N^2 / B + B)$

⇒ can be reduced to $O(N^2 / B + 1)$ because addition is only for the case when the whole matrix is smaller than one block and adding 1 will be enough.



Sufficient cache size:

the cache must be able to hold two tiles at once (for swapping)

In general case, each tile "involves" max $2B$ blocks

⇒ cache must keep $2B^2$ items for each tile

cache reads are by blocks

we have B rows with max 2 blocks in each row,

⇒ we will have $2B$ (one row) * B (# rows) = $2B^2$ values that must fit into the cache

⇒ cache size $M \geq 4B^2$ to keep two tiles

Tall-cache property: $M \in \Omega(B^2)$

if we view the cache as a rectangle whose rows are the blocks, the rectangle will be higher than it is wide.

Now, back to our cache-oblivious algorithm:

We will find the highest level of the recursion tree, at which the sub-problems correspond to tiles from the cache-aware algorithm.

optimal choice of tile size for cache-aware model was B

we will find the smallest i where sub-problem size $d = N / 2^i \leq B$.

Unless the whole input is small and $i = 0$, this implies $2d = N / 2^{i-1} > B$

i is the highest level where $N / 2^i \leq B$

previous level ($i-1$) has the double [sub]matrix size compared to current level

$2d > B \Rightarrow d > B/2$

⇒ $B/2 < d \leq B$

⇒ $d \in \Theta(B)$

Caching strategy:

[notes of this year] Caching strategy knows M and B !

⇒ it can choose the appropriate level i , where $N / 2^i \in \Theta(B)$.

Above level i , we cache nothing — this is correct, since we touch no items.

When we enter a node at level i , we load the whole sub-problem to the cache and compute the whole subtree in the cache.

all subsequent recursions will fit into the cache because the whole subproblem fits

⇒ it's $O(B)$ I/O operations for one subproblem

Doing this in all nodes of level i is equivalent to running the cache-aware algorithm.

⇒ requires $O(N^2/B + 1)$ block transfers in total, provided that the cache is tall enough.

If N is not a power of 2, all submatrices are square or almost square (lengths of sides differ by at most 1) and the asymptotics is the same.

Overhead in the cache-oblivious model can be reduced by stopping at some small subproblem size, e.g. 16×16 , rather than going down to 1×1 .

[Minor] State and prove the Sleator-Tarjan theorem on competitiveness of LRU.

Kate, [lecture notes](#) + [lecture video](#) (starting from 3:00)

Intuition: We would love for our least recently used (LRU) cache strategy to be about as good as an optimal perfect strategy up to a constant factor of difference. This is not possible, but we can prove that if we make our cache larger than that which is using a perfect strategy we can approximate its performance up to a constant factor. This constant will depend on the sizes of the caches we're analyzing.

Prerequisites:

[External memory model aka I/O model](#)

and [Cache models \(cache-aware and cache-oblivious\)](#)

Setup:

The main memory is divided into blocks, each identified by its address.

We are given a sequence of requests for particular blocks.

The strategy controls a cache of M block-sized slots.

Initially, each slot is either empty or it contains a copy of an arbitrary memory block.

Every time the strategy receives a new request:

if block is in cache: it points to a slot containing the requested block

if not: it's a miss and cache replaces the contents of one slot by the requested block

Cache efficiency aka cost of the strategy aka C: the number of cache misses.

So, we minimize C.

Optimal strategy (OPT):

knows the whole sequence of requests in advance

always uses the cache slot, whose block is needed farthest in the future (at best never)

cost: C_{OPT}

Online strategies - know only the current state of the cache and the current request

NB: history of last requests is useless (we do not do any ML here)))))

Least recently used (LRU) strategy:

keeps the cache slots sorted by the time of their last use

When the cache misses, it replaces the longest unused slot.

cost: C_{LRU}

Strategy STR is k-competitive if $C_{STR} \leq k \cdot C_{OPT}$

the cost of the strategy is bounded by $k \cdot C_{OPT}$, where k is the constant.

[comment] should be kind of "at most k times worse than optimal strategy" but they do not use this explanation for some reason

k - competitiveness ratio

We want to show that LRU is k-competitive for some constant k independent of M.

It's not possible in a straightforward way though:

for every online strategy, there is a request sequence which forces $T \geq \sim M \cdot C_{OPT}$

Theorem:

For every $M_{LRU} > M_{OPT} \geq 1$ and every request sequence, we have:

$$C_{LRU} \leq \frac{M_{LRU}}{M_{LRU} - M_{OPT}} \cdot C_{OPT} + M_{OPT}.$$

Reminder: M is the number of slots in the cache.

NB: we give larger size to LRU than to OPT because LRU is not k -competitive with the same cache size.

Example (and corollary):

if $M_{LRU} = 2M_{OPT}$:

$C_{LRU} \leq 2C_{OPT} + M_{OPT}$ (replace M_{LRU} with $2M_{OPT}$ in the above equation)

$\Rightarrow$ for long enough request sequences, $(2 + \epsilon)$ -competitive ratio.

We gave LRU a larger cache so we compare LRU on a cache of size M with OPT on a cache of size $M/2$. But when we change M by a constant factor, I/O complexity also changes by at most a constant factor. So using LRU does not change the asymptotics.

Theorem proof:

0. Split request sequence into epochs $E_0, E_1, \dots, E_k$ so that:

in each epoch except for E_0 number of misses equals to $M_{LRU} \Rightarrow C_{LRU} = M_{LRU}$

in E_0 , number of misses is at most $M_{LRU} \Rightarrow C_{LRU} \leq M_{LRU}$

Divide the sequence from the end:

count misses starting from the end

when # misses = M_{LRU} , put the epoch boundary

1. Epoch E_i for $i > 0$.

1.1. Case 1: All LRU's misses are different blocks.

$\Rightarrow$ epoch contains requests to at least M_{LRU} different blocks

because # misses = M_{LRU}

$C_{LRU} = M_{LRU}$ at this epoch

$\Rightarrow$ number of OPT misses is at least $M_{LRU} - M_{OPT}$

$C_{OPT} = M_{LRU} - M_{OPT}$ at this epoch

epoch contains requests to at least M_{LRU} different blocks

M_{OPT} of these blocks can be stored in OPT cache

reminder: $M_{OPT} < M_{LRU}$ (as a presupposition)

$\Rightarrow$ OPT cache just cannot store all those different blocks even if it's filled only with relevant blocks and there will be at least $M_{LRU} - M_{OPT}$ misses just because of cache size difference.

$\Rightarrow$ the ratio $\frac{C_{LRU}}{C_{OPT}} \leq \frac{M_{LRU}}{M_{LRU} - M_{OPT}}$

$\Rightarrow C_{LRU} \leq \frac{M_{LRU}}{M_{LRU} - M_{OPT}} \cdot C_{OPT}$, so we got the first part of the theorem

1.2. Case 2: LRU misses twice on the same block.

If LRU sorts the blocks by the last time of access (the oldest ones are at the end):

the first miss will put the block at the start of the block list in the cache

the second miss means that the block has gone off the list

⇒ between the misses, there were requests to at least M_{LRU} different blocks so that this block gets to the end of the list and then gets pushed away

so, it's the same as in 1.1:

⇒ epoch contains requests to at least M_{LRU} different blocks

⇒ number of OPT misses is at least $M_{LRU} - M_{OPT}$ (same as in 1.1)

⇒ the ratio $\frac{C_{LRU}}{C_{OPT}} \leq \frac{M_{LRU}}{M_{LRU} - M_{OPT}}$

⇒ $C_{LRU} \leq \frac{M_{LRU}}{M_{LRU} - M_{OPT}} \cdot C_{OPT}$, so we got the first part of the theorem

⇒ If this ratio holds for every E_i where $i > 0$, it holds for the whole sequence except for E_0

2. Epoch E_0

2.1. Case 1: the cache is empty at the start

⇒ epoch contains requests to at most M_{LRU} different blocks

if the cache is empty, all differing requests will be cache misses

in E_0 we have $C_{LRU} \leq M_{LRU}$ (by the construction of epochs)

⇒ $C_{OPT} \geq C_{LRU}$

again, just because OPT has smaller cache

OPT misses every time LRU misses because the cache is initially empty and OPT cache also “forgets” more frequently

⇒ the ratio $\frac{C_{LRU}}{C_{OPT}} \leq 1 \leq \frac{M_{LRU}}{M_{LRU} - M_{OPT}}$

the final fraction is at least 1 (by our presuppositions about cache sizes?)

2.2. Case 2: OPT starts with a non-empty cache

⇒ it can prevent up to M_{OPT} misses, making cost ratio greater than 1.

so we add up this M_{OPT} to the right-hand side of theorem inequality, thus raising (softening) our upper bound for C_{LRU} .

2.3. Case 3: LRU starts with a non-empty cache

⇒ it will be able to make fewer misses

⇒ the ratio $\frac{C_{LRU}}{C_{OPT}}$ will be even smaller which is what we want.

[Major] Describe hashing with chains and analyze its complexity. Define c -universal and k -independent systems of hash functions and provide constructions of such systems.

Prev: [Minor] Define c -universal and k -independent systems of hash functions. State and prove the theorem on the expected chain length in hashing with chains.

2023: TODO: Show examples of c -universal and k -independent systems for hashing of integers.

Jacob, this year's and last year's notes

Prereq: [hashing notation](#)

Intuitive explanation: c -universality and k -independence are families that basically tell us how much worse a function (in those families) is compared to a completely random function. c -universality tells us how much worse a function is in terms of how many more times (c) a function is likely to have collisions. Similarly, k -independence tells us how likely it is that a given set of elements is mapped to a specific set of buckets we pick, this tells us up to what degree each mapping is an independent event. The lower the c value the better and the higher the k value the better.

Let $\mathcal{H}$ be a family of functions $\mathcal{U} \rightarrow [m]$ we say that the family is c -universal for some $c > 0$ iff

$$\forall x, y \in \mathcal{U}, x \neq y: \Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{c}{m}$$

What this is saying is that we're only c times more likely to have a hashing collision between two distinct keys. **Remember: this is a statement about the family, not a singular function.** The function h we use in the statement is a function chosen at random from the family.

We pick h uniformly random from our family and we assume that we can evaluate it in $O(1)$ time for all values. We can also specify h to be h_a with parameter a , and thus choosing a function at random boils down to pick an a at random.

Theorem: Let $\mathcal{H}$ be a c -universal family of functions from $\mathcal{U}$ to $[m]$, $X = \{x_1, \dots, x_n\} \subseteq \mathcal{U}$ a set of numbers stored in the DS and $y \in \mathcal{U} \setminus X$ an item not stored in the DS, then we have:

$$\mathbb{E}_{h \in \mathcal{H}}[\#i: h(x_i) = h(y)] \leq \frac{cn}{m}$$

What does this mean? That means that the upper bound on elements that will collide with y is cn/m . The proof is very intuitive and concise in the notes so I will just paste it here:

Proof: Let h be a function picked uniformly at random from $\mathcal{H}$. We introduce indicator random variables

$$A_i = \begin{cases} 1 & \text{if } h(x_i) = h(y), \\ 0 & \text{otherwise.} \end{cases}$$

Expectation of zero-one random variables is easy to calculate: $\mathbb{E}[A_i] = 0 \cdot \Pr[A_i = 0] + 1 \cdot \Pr[A_i = 1] = \Pr[A_i = 1] = \Pr[h(x_i) = h(y)]$. Because $\mathcal{H}$ is universal, this probability is at most c/m .

The theorem asks for the expectation of a random variable A , which counts i such that $h(x_i) = h(y)$. This variable is a sum of the indicators A_i . By linearity of expectation, we have $\mathbb{E}[A] = \mathbb{E}[\sum_i A_i] = \sum_i \mathbb{E}[A_i] \leq \sum_i c/m = cn/m$. $\square$

We basically calculate the expectation of a collision on a per item basis. We can then simply by linearity move the sum out of the expectation of the sum and we get the sum of expectations, which is cn/m as we were hoping for.

Corollary On the complexity of hashing with chaining:

We can use the c -universality of a family to prove some facts about hashing with chaining. (Quick clarification here: hashing with chaining simply means that if there is a collision we store all values with a matching mapping to the same bucket using a linked-list or some other auxiliary DS).

Let's say we have $x_1, \dots, x_n$ items stored in our hash table and we wanna operate over some y .

FIND(y) complexity: for finding y we need to basically go over all items that have a collision with $h(y)$, which by the above definition we know would **at most** be cn/m items. So, if our m is $\Omega(n)$ (if we have asymptotically more buckets than elements) we would end up with constant time.

INSERT(y) complexity: inserting takes constant time but we need to check if the item doesn't exist first. To do this, it's the equivalent of a **FIND(y)** operation.

DELETE(y) complexity: Just like above, we need the equivalent of **FIND(y)** in order to find the element and we remove it in constant time.

Because of this, **if we pick an m that is in the order of n , i.e. $\Omega(n)$, then we can say that hashing with chaining in a c -universal family takes $O(1)$ for all operations.**

One caveat to this is however, that if we do not know how many elements will be inserted into the table, we would probably just pick some initial m and then once we reach a certain threshold, we can resize our hash table and we rehash everything. This strategy is similar to what we see for [flexible arrays](#).

k-independence: There is another stronger property for families of hashing functions. This is (k, c) -independence. We **define** it as such:

Let $\mathcal{H}$ be a family of hashing functions from $\mathcal{U}$ to $[m]$. The family is (k, c) -independent for a k , $1 \leq k \leq |\mathcal{U}|$ and a $c > 1$ if and only if for every tuple $x_1, \dots, x_k$ of **distinct** elements of the universe $\mathcal{U}$ and every k -tuple $a_1, \dots, a_k$ from $[m]$ the following holds

$$\Pr_{h \in \mathcal{H}}[h(x_1) = a_1 \wedge \dots \wedge h(x_k) = a_k] \leq \frac{c}{m^k}$$

What does this mean? It means that if we have a set of distinct elements and a set of buckets that we can map to, the probability of the elements being assigned *specifically* to that set of buckets must not exceed c/m^k , meaning each assignment is independent from the previous one. Let's use $k = 2$ as an example:

If we have elements x_1 and x_2 and we have *specific* buckets $a, b \in [m]$, then x_1 will be assigned to some bucket. The probability of that bucket being a_1 is $1/m$ for a perfectly random function. *And* the probability of x_2 being assigned to b is **also** $1/m$. If these events truly are independent, then the probability of them both happening is $1/m^2$. Note that a and b need not be distinct, but x_1 and x_2 have to be. We generalize this example to k values to be mapped and we get our definition.

Note: If we don't care about c we simply call this property k -independence.

Some observations about k -independence:

1. If a family is k -independent for $k > 1$, then it is also $(k - 1)$ -independent.
2. If a family is $(2, c)$ -independent, then it is also c -universal.

We proved these [here](#), in case you're curious.

[Minor] Describe a system of linear hash functions. Prove that it is a 2-independent system from $\mathbb{Z}_p$ to $[m]$.

Andy, Resources: [Martin Mareš's Notes](#) (p65-66), [Video](#) (proof starting at 0:43:26, with introduction to linear hash functions starting at 0:18:36)

Prereq: [c-universality and k-independence](#)

Overview: We will create linear hash functions that map from a universe with size some prime number to the same universe, then reduce these values to a smaller universe with size equal to the number of buckets. To show 2-independence, we create two separate random events that are equivalent to the random parameters of the hash function, and show for each that the probability of reduction to the same bucket is at most $\frac{2}{m}$, which makes the conjunction of the probabilities at most $\frac{4}{m^2}$, and thus (2,4)-independent.

Definition: $\mathcal{L} = \{h_{a,b} \mid a, b \in \mathbb{Z}_p\}$ from $\mathbb{Z}_p$ to $[m]$ is a system of linear hash functions for prime p and $m \leq p$, where $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$.

Theorem: The family $\mathcal{L}$ is (2,4)-independent. (Reminder: (k,c)-independence means that $\Pr_{h \in \mathcal{H}}[h(x_1) = a_1 \wedge \dots \wedge h(x_k) = a_k] \leq \frac{c}{m^k}$, that each assignment to a bucket is independent of each other up to some value of k .)

Proof: We need to show that, if $x_1 \neq x_2$, and for $i, j \in [m]$, $\Pr[h_{a,b}(x_1) = i \wedge h_{a,b}(x_2) = j] \leq \frac{4}{m^2}$ for a random choice of parameters a and b .

First let $r = (ax_1 + b) \bmod p$, and $s = (ax_2 + b) \bmod p$. These are the buckets that x_1 and x_2 are mapped to in $\mathbb{Z}_p$, which will then be reduced to the set of buckets $[m]$ by the modulo operation. Since we are in a field, these two equations have a unique solution, and so picking parameters a and b uniformly at random is equivalent to picking r and s uniformly at random.

Then we have that $\Pr[h_{a,b}(x_1) = i \wedge h_{a,b}(x_2) = j] = \Pr[r \equiv i \bmod m \wedge s \equiv j \bmod m]$. Since the choices of r and s are independent, this is the same as the product of the independent probabilities $\Pr[r \equiv i \bmod m] \cdot \Pr[s \equiv j \bmod m]$, and we can take the two events separately.

For r , in each of the $\lceil \frac{p}{m} \rceil$ intervals in the universe, there is at most one r such that $r \equiv i \bmod m$. r has p possibilities, so the probability of congruence is $\frac{\lceil p/m \rceil}{p} \leq \frac{p+m-1}{mp} \leq \frac{2p}{mp} = \frac{2}{m}$. The same is true for s , and so the product of these probabilities is at most $\frac{4}{m^2}$, which is exactly what is required to show the proof. ■

[Minor] Construct a k -independent system of hash functions from $\mathbb{Z}_p$ to $[m]$.

Jacob, last year's and this year's notes. Last year's lecture ([51 mins in](#)), [Martin Mareš's Notes](#)(pg 69)

Intuitive explanation: as in the previous question, when [forming a system of linear hash functions](#) we used linear functions to achieve 2-independence, now we can simply expand this concept and use polynomials of higher degrees to achieve higher k -independence properties. Namely, we will use $(k - 1)$ degree polynomials to achieve k -independence.

We first **define** a new family of functions as such:

For any field $\mathbb{Z}_p$ and any $k \geq 1$ we define a family of polynomial hash functions

$$\mathcal{P}_k = \{h_{\vec{t}} | \vec{t} \in \mathbb{Z}_p^k\} \text{ from } \mathbb{Z}_p \text{ to } \mathbb{Z}_p \text{ where } h_{\vec{t}}(x) = \sum_{i=0}^{k-1} t_i x^i \text{ where } \vec{t} \text{ is a } k\text{-tuple indexed from } 0.$$

(Again, t should be bolded but latex support is terrible here so using an arrow instead)

So what does this definition mean? We have a family of functions where the function maps from one field to itself (this time we won't be reducing the universe further by doing modulo m , not yet anyway). Our function parameters will be a k -tuple as opposed to just (a, b) like before, since this is a generalization of the above. The function will be computed as a polynomial of degree $k - 1$.

Lemma: Family $\mathcal{P}$ is $(k, 1)$ -independent.

The lemma is basically what we set out our goal to be at the start, get a family that generalizes the above concepts. Now, for the proof.

Proof: Let $x_1, \dots, x_k \in \mathbb{Z}_p$ be distinct items and $a_1, \dots, a_k \in \mathbb{Z}_p$ buckets.

If we have k points in a plane, there exists only one polynomial of degree $(k - 1)$ that connects all points (by [Lagrange's interpolation](#)). So what does this mean?

That $\forall i: h_{\vec{t}}(x_i) = a_i$ determines a unique polynomial that fits the equation. **So this means that a set of points and their specific correspondent buckets determines the parameters of h . Thus:**

$$\Pr[\forall i: h_{\vec{t}}(x_i) = a_i] = \frac{1}{p^k}$$

Why? Because only one polynomial complies with the condition and we have p^k total possibilities for a set of parameters for the polynomial.

[Zhenya] The number of a_i we need is k . And each $a_i \in [0, p - 1]$ (because it's from the field $\mathbb{Z}_p$), thus we have $p * p * \dots * p = p^k$ possible arrangements of a 's. We need to get 1 in particular, so the probability is $\frac{1}{p^k}$.

But that equation above is exactly what being $(k, 1)$ -independence means, so we have proven our theorem.

Final note: We are mapping a field to a field here, if we wanna have $\mathcal{P} \bmod m$ then we need to pick a large enough p , i.e. $(p \geq 2km)$ to get a $(k, 2)$ -independent family, see [lemma k](#). The complexity however depends on k and is $\Theta(k)$ since we have to pick a function and evaluate it.

[Minor] Construct a 2-independent system of hash functions for hashing of strings of length at most L over an alphabet $[a]$ to a set of buckets $[m]$.

Jacob, this year's and last year's notes

[Zhenya: this is a quite useful [video](#) from the last year practicals]

Prereq: [composition lemma](#), [linear hash function family](#)

Intuitive explanation: We want a system that can hash strings in an efficient manner. A good choice for this would be rolling hashes from polynomials. This family of functions has functions that map from $\mathbb{Z}_p^d$ to $\mathbb{Z}_p$. This means that we still need to reduce this further to $[m]$, we will do this by taking advantage of the composition lemma and picking a $(2, 4)$ -independent family. This will result in a 2-independent family. This resulting family could then be used to hash strings by simply padding strings to the max length (which will determine what field we use for the functions in the family).

We will be defining a family of rolling hashing functions as follows:

For a prime p and vector size d , we define the family of polynomial hash functions

$$\mathcal{R} = \{h_a | a \in \mathbb{Z}_p\} \text{ from } \mathbb{Z}_p^d \text{ to } \mathbb{Z}_p \text{ where } h_a(x) = \sum_{i=0}^{d-1} x_{i+1} \cdot a^i = x_1 + x_2 a + x_3 a^2 + \dots + x_d a^{d-1}$$

We see that these functions use the parameters as the exponentiated part of the product. We will see why this is useful later on.

Theorem: The family $\mathcal{R}$ is d -universal. Additionally, a function can be picked from $\mathcal{R}$ and evaluated for a given vector in $\Theta(d)$ time.

Proof: Let us consider two vectors $\vec{x} \neq \vec{y}$ and a hash function h_a chosen at random from $\mathcal{R}$.

To prove d -universality we analyze when a collision might happen. This would be

$$\sum_i (\vec{x} - \vec{y})_{i+1} \cdot a^i = 0, \text{ which we get by simply modifying the definition of collision. So this}$$

collision happens when a^i is a root of the $(\vec{x} - \vec{y})$ polynomial. But a polynomial of dimensionality d can only have d roots, so the probability of a^i being one is d/p , this already implies d -universality.

More formally we can say that $\Pr[\sum_i (\vec{x} - \vec{y})_{i+1} \cdot a^i = 0] \leq \frac{d}{p}$.

If we use our composition lemma and pick $\mathcal{L}$, meaning our new family would be $\mathcal{R} \circ \mathcal{L}$, we would have a $(2, 5/2)$ -independent family given a prime p and a number of final buckets $[m]$, such that $p \geq 4dm$.

[Kate] calcs for composition lemma:

$$c = d$$

$$d_{\text{in lemma}} = 2 \text{ (}\mathcal{L} \text{ is } (2, 2)\text{-independent for } p \geq 4m)$$

$$c' = \left(\frac{cm}{p} + 1\right) \cdot d_{\text{in lemma}} = \left(\frac{dm}{4dm} + 1\right) \cdot 2 = \left(\frac{1}{4} + 1\right) \cdot 2 = \frac{5}{2}$$

So, $\mathcal{R} \circ \mathcal{L}$ is $(2, c')$ -independent, where $c' = 5/2$

These rolling hashing families are quite useful when applying the *Rabin-Karp string search algorithm*. This basically addresses the problem of a string (our needle) search over a long string (our haystack). Instead of doing character by character comparisons across the long string, we simply hash the needle and hash portions of the long string of the same length of the needle and then slide the window. If the hashes for the needle and the window match, then we do the char-by-char comparison (because remember, **we can have false positives**), otherwise we just keep going.

For this algorithm to be fast though, each hashing of the window needs to be able to be done in $O(1)$ time. This is where the rolling hashing comes in. Let's compare how the hashing looks for two time steps of the sliding window, at times j and $j + 1$:

$$H_j = h(\sigma_j, \dots, \sigma_{j+d-1}) = \sigma_j a^{d-1} + \sigma_{j+1} a^{d-2} + \dots + \sigma_{j+d-1} a^0$$

$$H_{j+1} = h(\sigma_{j+1}, \dots, \sigma_{j+d}) = \sigma_{j+1} a^{d-1} + \sigma_{j+2} a^{d-2} + \dots + \sigma_{j+d} a^0$$

We can see that $H_{j+1} = aH_j - \sigma_j a^d + \sigma_{j+d}$. This can be done in $O(1)$ time if we precompute a^d .

Hashing strings: We can use the above defined family to hash strings very efficiently. To handle our length L , **we can simply pad shorter strings to length L** . If we pad with 0, then those padded values would simply not affect anything.

We also need to pick a p such that the length of the alphabet k is strictly smaller than p . If we are able to pick a $k \ll p$, then we can also get away with **encoding several characters into one slot** like so:

$$\boxed{x_0 \mid x_1 \mid x_2}$$

$$x_0 + kx_1 + k^2x_2 < p$$

Final note: there is also a modulo m lemma which allows us to further reduce a family of functions $\mathbb{Z}_p^d$ to $\mathbb{Z}_p$ to $[m]$ if we simply add modulo m . However, using the [composition lemma](#) we get stronger properties, namely that we can keep the c' -universality from the lemma to be smaller as opposed to lemma M or K.

[Minor] Describe a system of hash functions based on scalar products. Prove that it is a 1-universal system from $\mathbb{Z}_p^k$ to $\mathbb{Z}_p$.

Andy, Resources: [This year's notes](#) (p4), [Martin Mareš's notes](#) (p71), [Video](#) (starting at 1:07:11)

Definition: For a prime p and vector with size $k \geq 1$, the family of scalar product hash functions is $S = \{h_{\vec{t}} \mid \vec{t} \in \mathbb{Z}_p^k\}$ from $\mathbb{Z}_p^k$ to $\mathbb{Z}_p$, where $h_{\vec{t}}(\vec{x}) = \vec{x} \cdot \vec{t}$.

Theorem: S is 1-universal (see [above](#) for definition of c-universality, but briefly it is the probability that a collision between two keys occurs with probability at most $\frac{c}{m}$). A function can be picked at random from S in time $\theta(k)$ and evaluated in the same time.

Proof: Consider two vectors $\vec{x}, \vec{y} \in \mathbb{Z}_p^k$. Let d be a coordinate for which $x_d \neq y_d$. Since an inner vector product does not depend on the ordering of the components, we can renumber the components of x and y such that $d = k$. (This is just to simplify the proof, and does not affect generality)

Then we can say that for a random choice of parameter $\vec{t}$, the probability of a collision is

$$\Pr_{h \in S}[h_{\vec{t}}(\vec{x}) = h_{\vec{t}}(\vec{y})] = \Pr[\vec{x} \cdot \vec{t} = \vec{y} \cdot \vec{t}] = \Pr[(\vec{x} - \vec{y}) \cdot \vec{t} = 0] = \Pr[(x_k - y_k) \cdot t_k = -\sum_{i=1}^{k-1} (x_i - y_i)t_i]$$

Since $x_k - y_k \neq 0$ and we are in a [field](#), the final equation has a unique solution of t_k . And since $t_1, \dots, t_{k-1}$ is picked uniformly in random, the right hand side is also uniformly random (except if $\forall i < d: x_i = y_i$, in which case still $\Pr(t_k = 0) = \frac{1}{p}$).

That t_k is the unique solution of the equation has probability $\frac{1}{p}$. ■

[Kate adding screenshots about the reduction to modulo m]

What if the field is too large and we want to reduce the result modulo some m ? To use Lemma **M**, we need a 2-independent family, but $\mathcal{S}$ is merely universal.

We can use Lemma **G** and compose $\mathcal{S}$ with the family $\mathcal{L}$ of linear functions from $\mathbb{Z}_p$ to $[m]$. As $\mathcal{L}$ is $(2, 4)$ -independent, $\mathcal{S} \circ \mathcal{L}$ is $(2, 8)$ -independent. If $p \geq 4m$, $\mathcal{L}$ is $(2, 2)$ -independent and $\mathcal{S} \circ \mathcal{L}$ $(2, c')$ -independent for $c' = (1 + 1/4) \cdot 2 = 5/2$.

Alternatively, we can add an additive constant to the scalar product:

Definition: For a prime p , vector size $d \geq 1$, we define the family of scalar product hash functions $\mathcal{S}' = \{h_{\mathbf{t}, \beta} \mid \mathbf{t} \in \mathbb{Z}_p^d, \beta \in \mathbb{Z}_p\}$ from $\mathbb{Z}_p^d$ to $\mathbb{Z}_p$, where $h_{\mathbf{t}, \beta}(\mathbf{x}) = \mathbf{t} \cdot \mathbf{x} + \beta$.

Theorem: If $p \geq 4m$, the family $\mathcal{S}' \bmod m$ is $(2, 2)$ -independent. A function can be picked at random from the family in time $\Theta(d)$ and evaluated in the same time.

Proof: By exercise 6, the family $\mathcal{S}'$ is $(2, 1)$ -independent. By Lemma **K**, $\mathcal{S}' \bmod m$ is $(2, 2)$ -independent. □

[Minor] Describe the cuckoo hashing and state the theorem on its complexity (without proof).

Toni, lecture notes by Martin Mareš

Cuckoo hashing

- Two functions $f, g: U \rightarrow [m]$
- each bucket contains at most one item
- an item x can be located only in buckets $f(x)$ and $g(x)$
- lookup checks only these two buckets $\rightarrow O(1)$ in the worst case

Insert operation:

- looks inside both buckets
- if either is empty, put the new item there
- otherwise, kick out an item from one of the buckets and add the new item there instead
- but now, we have to find a place for the kicked out item
 - we do that but using the other hash function
 - hashing function which is different from the one used for initial placing of the current kicked out item.
 - this may lead to kicking out some other item, etc.
- After roughly $\log n$ attempts ("insertion timeout"), we give up, pick two new hash functions f and g , and we rehash everything from scratch

Theorem: Let $\epsilon > 0$ be a fixed constant. Suppose that $m \geq (2 + \epsilon) n$, insertion timeout is set to $\lceil 6 \log n \rceil$, and f, g chosen at random from a $\lceil 6 \log n \rceil$ -independent family. Then the expected time complexity of INSERT is $O(1)$, where the constant in O depends on ϵ .

Note:

- Setting the timeout to $\lceil 6 \log m \rceil$ also works
- Tabulation hashing suffices even though it is 3-independent, and not 4-independent

[Minor] Describe hashing with linear probing and give overview of results on its complexity.

Prev: [Major] Describe and analyze hashing with linear probing.

Jacob, last year's and this year's notes

Resources: The whole Martin Mareš [video](#), Martin Mareš [lecture notes](#)

Intuitive explanation: When assigning items to a bucket in a hash map, we could simply restrict each bucket to only hold 1 item, this is called open addressing. This will make it so a representation of the map is just an array of size m and a hashing function that maps a value to $[0, m - 1]$. But what if there are collisions? Well, we basically follow a probing sequence until we find an empty cell and that becomes the key's new home. In linear probing the probing sequence is simply adding 1 to the previous attempt. Insertions are pretty straight forward and deletes require the "tombstone" trick. This approach might sound naive but if we keep the array sparse enough and pick a good enough hashing function, it has nice performance.

Formal description:

First we fix a hashing function h from the universe to $[m]$. This would make the probing sequence for x be $h(x)$, $h(x) + 1$, $h(x) + 2$, ... and taking modulo m .

Formally, linear probing would be defined as:

We iterate over an i as we evaluate our hash function h , where i represents the attempt number. Our *first* attempt, i.e. the initially desired bucket is when $i = 0$. Formally:

$$h(x, i) = (h(x) + i) \bmod m$$

with a probing sequence of

$$h(x, 0), h(x, 1), \dots, h(x, m - 1)$$

With $h(x, 0)$ being the primary location, meaning the intended location for the item.

What this means is that we first check whether the cell at $h(x) \bmod m$ is free. If not then we try again with $(h(x) + 1) \bmod m$. We repeat this until we find an empty cell.

Linear probing can be useful because it is quite **cache-friendly**, since we're accessing our array sequentially. However, we need to make sure to have a **sparse enough array**, (i.e. $m \geq 3n$ for our intents and purposes but generally $m = (1 + \epsilon)n$), as this method can lead to clusters of occupied cells (runs). We're trading some space to guarantee good performance.

SIDE NOTE: there is also the possibility to do open addressing with multiple hashing functions. For instance, double hashing would look like this:

$$h(x, i) = h(x) + i \cdot g(x) \bmod m \text{ where } h, g \text{ are chosen uniformly randomly from a family}$$

Operations:

FIND(x): We simply follow the probing sequence until finding either the x or an empty cell and returning the appropriate value accordingly. Finally, if we reach $h(x, m - 1)$ (i.e. having made m attempts) and it is not the cell we are after, we return *false*.

INSERT(x): Just like with *FIND* we follow the probing sequence until we find an empty cell. We then insert the element in this empty cell.

DELETE(x): For deletions, we cannot simply remove an item as that could interrupt the linearity of our system. This would result in false negatives. So, what we do is we **replace items to be deleted with “tombstones” instead of emptying the cell**. This will signal to us during the *FIND* and *INSERT* that we should continue along the probing sequence but that an element at that cell was deleted. This tombstone cell can be used for insertions but if we’re running a *FIND* we should continue along the path. Also, we can rehash the whole thing after some time.

Complexity: We define the complexity for these operations as **the number of probes** in the *FIND* operation with respect to h and the number of tombstones.

Now we have some claims regarding the complexity depending on the h function we pick:

Let $m \geq (1 + \epsilon)n$ then the number of probes in *FIND* is:

- $\mathcal{O}(1/\epsilon^2)$ for a completely random hash functions
- $\mathcal{O}(1/\epsilon^{13/6})$ for hash function chosen from a 5-independent family
- $\Omega(\log n)$ for at least one 4-independent family
- $\Omega(\sqrt{n})$ for at least one 2-independent family
- $\Omega(\log n)$ for multiply-shift hashing
- $\mathcal{O}(1/\epsilon^2)$ for tabulation hashing

(Screenshot from notes because Google docs latex support is ass)

First two and the last one are good, others are bad. We want constant access time after all.

Some notes: the last result is surprising because it’s only 4-ind, but its additional properties are quite nice and allow for the complexity to be good. 4-ind and below on its own is definitely not enough.

Theorem: we will prove a slightly simplified version of the first result stated as such:

*Let $m \geq 3n$ be a power of 2, $h: U \rightarrow [m]$ totally random function, $x \in U$, then the expected number of probes in *FIND* is $\mathcal{O}(1)$.*

What are the simplifications? We bound m , we also simplify to result to $\mathcal{O}(1)$ from $\mathcal{O}(1/\epsilon^2)$.

We will also further simplify by setting $m = 3n$, even though this is not possible because m is a power of 2. But we can do this without loss of generality since the complexity can only get better as n decreases.

Proof layout:

1. Some definitions
2. Prove that we can bound how likely critical blocks are
3. Prove that long runs are unlikely *because* they *have* to contain a large critical block
4. Lay out the previous point in more probabilistic terms
5. Putting it all together (remember, we are trying to prove that basically the length of runs is bounded by a constant thus keeping our complexity in $O(1)$).

Proof:

We will denote items in the table as $x_1, \dots, x_n$

First, we need to define something that will be useful later in the proof and that is the standard way of estimating tail probabilities over a distribution, i.e. how likely values in the tail of a distribution are: The Chernoff bound to the right tail.

Let $X = X_1 + \dots + X_k$ where X_i are independent binary random variables. Also, $\mu = E[X]$ (the expectation, i.e. mean) and $c > 1$. We say:

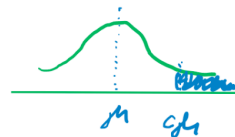
$$\Pr[X > \mu c] \leq \left(\frac{e^{c-1}}{c^c}\right)^\mu$$

More details below:

Then (Chernoff b.): Let $X = X_1 + \dots + X_k$ where X_i 's are independent binary random variables, $\mu = E[X]$ (mean), $c > 1$. Then

$$\Pr[X > c\mu] \leq \left(\frac{e^{c-1}}{c^c}\right)^\mu$$

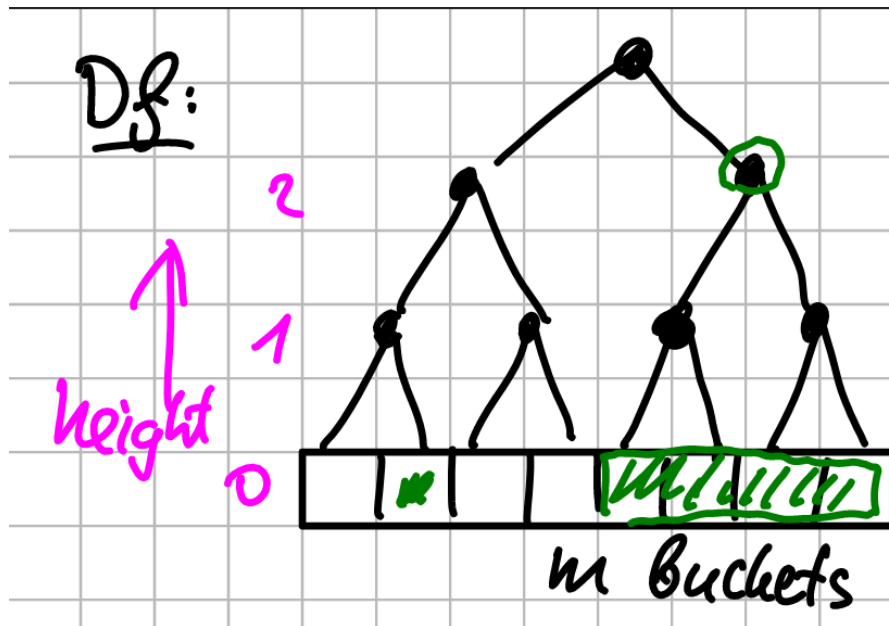
↗ right tail =



We also use: $\Pr[\bigcup_{i=1}^k A_i] \leq \sum_{i=1}^k \Pr[A_i]$ for any (countably many) events A_i , not necessarily indep.
(union bound)

and: (*) $E[X] \leq \sum_{i=1}^k \max I_i \cdot \Pr[X \in I_i]$ for a random variable X s.t. $\text{rang}(X) = I_1 \dot{\cup} \dots \dot{\cup} I_k$.
range ↗ ↘ disjoint union

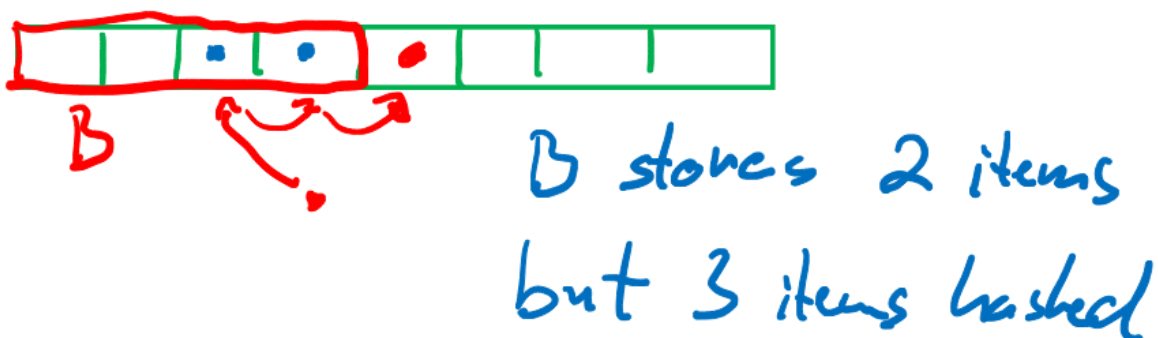
We also define *blocks*. We build binary trees over our array as pictured below.



A block is the interval of 2^t buckets below a node, where t is the height of the node. In the above illustration, a block is highlighted on the array and its reference node is highlighted.

Definition: A **block** B is **critical** if more than $2/3 \cdot 2^t$ items are hashed to B .

IMPORTANT NOTE: please remember that the definition here is that the items need to be *hashed* to B . If an item is in a block due to a long run but was initially hashed to another block, it doesn't count towards the limit. Conversely, if an item is hashed a block but the block could not store, the item *does* count towards the limit mentioned above. An illustration follows:



Lemma 1: Let B be a block of size 2^t . Then $\Pr_h[B \text{ is critical}] \leq (e/4)^{2^{t/3}}$.

What is this saying? Well, the probability of a block being critical should decrease as the size increases.

Also, we will rewrite $(e/4)^{1/3} \approx 0.879$ as q simply. Thus $\Pr_h[B \text{ is critical}] \leq q^{2^t}$.

Proof of lemma 1:

We define X_i as 0 if the item is not mapped to B and as 1 if the item is hashed to B .

(Note that x_i is different from X_i , the former being elements of the table.)

This means that we can get the number of hashed elements to B by just $\sum_i X_i$.

So now we have the distribution and we want to calculate the tail of that distribution. But in order to use the Chernoff bound, we need the mean.

$$E[X_i] = 0 \cdot \Pr[X_i = 0] + 1 \cdot \Pr[X_i = 1]$$

$$E[X_i] = \Pr[X_i = 1] = \Pr[h(x_i) \in B] = 2^t/m$$

Basically, the expectation basically just ends up being a ratio between the size of the block and m **because our h is a perfectly random hashing function.**

So now we can get the mean by summing all expectations and we get:

$$\mu = E[X] = \sum_i E[X_i] = n \cdot 2^t/m$$

And, by our assumption, we have $n = m/3$, so we get that $\mu = 1/3 \cdot 2^t$.

Now we just apply the Chernoff bound:

$$\Pr[B \text{ is critical}] = \Pr[X > 2/3 \cdot 2^t] = \Pr[X > 2\mu]$$

By using Chernoff bound with $c = 2$ (since our definition of critical is $2/3$ and our mean is $1/3$) we get

$$\leq \left(\frac{e^1}{2^2}\right)^\mu = \left(\frac{e}{4}\right)^{2^t/3}$$

Thus proving our lemma.

Lemma 2: Let's deal with runs now.

Let R be a run of length $\geq 2^{l+2}$, $B_0, \dots, B_3$ as the first blocks in the run of size 2^l intersecting R .

Then at least one of these 4 blocks is critical.

Our lemma is basically saying that long runs have a large critical block in them. We already have a bound for critical blocks by lemma 1, so the result of this lemma is that we are saying that long runs are unlikely.

Proof of lemma 2:

What is a run specifically? It's the maximal number of consecutive full buckets. Please note that this doesn't mean a run of maximal length, but rather that the run cannot be extended, i.e. the run has empty cells surrounding it.

Observation: Unlike with blocks, an item is hashed to a run iff it is stored in the run.

So now we determine what the size of the intersection of each block is with the run R .

$|B_0 \cap R| \geq 1$; $|B_{1..3} \cap R| = 2^l$, this is because the first block might not be fully overlapping with the run but it *at very least* intersects across one item. The rest of the blocks overlap completely because the size of the run is at least 2^{l+2} .

So the size of the overlap is $\geq 3 \cdot 2^l + 1$. BUT if none of the blocks are critical, then that means the number of items would be $\leq 4 \cdot 2/3 \cdot 2^l = 8/3 \cdot 2^l$. But this is a contradiction with the size of the overlap we calculated. Thus, at least one block needs to be critical. Thus proving our lemma.

Lemma 3: Let R be a run containing $h(x)$ and $|R| \in [2^{l+2}, 2^{l+3})$, then at least one of the following blocks is critical:

1. Either the block containing $h(x)$
2. One of the 8 blocks before that block
3. One of the 3 blocks after that block

Proof of lemma 3:

The size of R is bounded between $4 \cdot 2^l$ and $8 \cdot 2^l$. This means that R intersects at most 9 blocks.

From this we can just apply the previous lemma. Why? Well, the start of the run is *at most* 8 blocks before the block containing $h(x)$. And we know from the previous lemma that one of them has to be critical. And if the block containing $h(x)$ is at the very start of the run, then we know that block or one of the next 3 needs to be critical. Thus proving our lemma.

Corollary of the previous lemma:

If we have a run R containing $h(x)$ then $\Pr[|R| \in [2^{l+2}, 2^{l+3})] \leq 12 \cdot q^{2^l}$.

This is a simple corollary from the previous lemma. We simply use the union bound, i.e. the $\Pr[A \cup B] = \Pr[A] + \Pr[B]$. So we simply consider that any of the 12 blocks needs to be critical using the q from the first lemma.

Concluding the proof:

We now want to find the expected length of a run. With a run R containing $h(x)$,

$E[|R|] = \sum_k k \cdot \Pr[|R| = k]$ but we can split $|R|$ in intervals such that we can use our last

lemma to decompose the sum such that $|R| = [0, 3] \cup [4, 7] \cup \dots \cup [2^{l+2}, 2^{l+3})$:

$$\mathbb{E}[|R|] \leq 3 \cdot \Pr[|R| \leq 3] + \sum_{\ell \geq 0} 2^{\ell+3} \cdot \Pr[|R| \in [2^{\ell+2}, 2^{\ell+3}]].$$

We bound the former probability simply by 1 and use the previous corollary to bound the latter probability:

$$\mathbb{E}[|R|] \leq 3 + \sum_{\ell \geq 0} 2^{\ell+3} \cdot 12 \cdot q^{2^\ell} = 3 + 8 \cdot 12 \cdot \sum_{\ell \geq 0} 2^\ell \cdot q^{2^\ell} \leq 3 + 96 \cdot \sum_{i \geq 1} i \cdot q^i.$$

Since the last sum converges for an arbitrary $q \in (0, 1)$, the expectation of $|R|$ is at most a constant. This concludes the proof of the theorem. $\square$

What this boils down to is we bound the expected length of the run R to a constant value. We take advantage of the previous lemmas we have proven and the fact that we expanded the sum to fit nicely with our previous lemma.

The first part is: simply take the probability of the length of the run being of length 3 or less and we bound that from above to simply 1. We're bounding from above so it shouldn't matter too much that we do this.

The rest of the lengths we can bound by our previous lemma. Remember that our previous lemma (really its corollary) was that we bound the probability of really long runs to something. That something is q^{2^l} . We then just take every constant out of the sum and are left with $\sum_{i \geq 1} i \cdot q^i$. But we already know what q is and we know it converges to some value < 1 . So the expectation of the length of runs is bounded by a constant, which means that long runs are very unlikely, this doesn't even depend on how far up we go up in levels of our "imaginary" binary tree built. This all comes back to the fact that we determined that critical blocks are not very likely and they get less and less likely the longer they get.

[Minor] Describe and analyze the Bloom filter.

by Toni, source: lecture notes by Martin Mareš + video by Martin Mareš

Bloom filters are a family of probabilistic data structures usually used for testing whether a given element is a member of a set. Therefore, they are very approximate representations of sets in a small, bounded amount of memory. Why are they probabilistic? Because for a given element, operation "MEMBER" in Bloom filter can tell us either of two things:

- NO: Element is *definitely not in* the set
- YES: Element is *possibly in* the set

Because we cannot be sure that an element is in the set, Bloom filter can produce false positives: it is possible that Bloom filter can say that an element is in the set when it actually isn't. However, false negatives never occur since we deterministically know if an element is not in the set. In a sense, Bloom filters will make mistakes up to some allowed probability, which we can try to minimize at the expense of making the data structure larger.

Simple Bloom filter

Bloom filters operate with bit arrays. Consider a bit array B and a hash function $h: U \rightarrow [m]$ which serves as an index to an array B . Hash function will be 1-universal for simplicity. In the beginning all bits in B are 0.

Two operations:

- INSERT(x): $B[h(x)] \leftarrow 1$
- MEMBER(x): YES iff $B[h(x)] == 1$

Now suppose we have already inserted some n items. If we query for an element x_i ($1 \leq i \leq n$), it will always return YES. But what if we query for some y which is different from all x_i 's? Bloom filter can return YES and we get a false positive if y falls to the same bucket as one of the existing x_i 's! Probability of that false positive answer is:

$$Pr_h[h(y) = h(x_i)] \leq 1/m$$

By union bound (considering all x_i 's), the probability that $h(y) = h(x_i)$ for at least one i , is at most n/m :

$$\text{Petr Gregor's notes: } Pr_h[y \text{ is FP}] = Pr[\exists i h(y) = h(x_i)] \leq \sum_{i=1}^n Pr[h(y) = h(x_i)] \leq \frac{cn}{m}$$

[Kate] I think it's the [theorem](#) about collisions for c -universal hash families.

We can ask an inverse question: how large filter do we need in order to push error probability under some $\epsilon > 0$? $\lceil n/\epsilon \rceil$ bits would suffice.

Multi-band filters

This is an improvement of the previous approach where we have multiple filters in parallel and with this we can achieve the same error probability in smaller space. We choose k hash functions $h_1 \dots h_k$ where $h_i: U \rightarrow B_i[m]$ (i -th array of m bits). Each pair (B_i, h_i) is called a

band of the filter. Now inserting an element means that we insert it to all bands. Operation “MEMBER” now returns YES only if all bands answer YES.

What’s the error probability now? Suppose we set $m = 2n$, so that each band gives a false positive with probability at most $1/2$. The whole filter gives a false positive if all bands gave false positives, which happens with probability $(\frac{1}{2})^k = 2^{-k}$ (if hashing functions were chosen independently). This proves the theorem:

Theorem: Let $\epsilon > 0$ be the desired error probability and n the maximum number of items in the set. The k -band Bloom filter with $m = 2n$ and $k = \lceil \log(1/\epsilon) \rceil$ gives false positives with probability at most ϵ . It requires $2n \lceil \log(1/\epsilon) \rceil$ bits of memory and both INSERT and LOOKUP run in time $O(k)$

Optimizing parameters: we can fine-tune parameters of multi-band filter to improve memory consumption. Optimization problem: given a memory budget of M bits, set the parameters m and k such that the filter fits in memory and the error probability is minimized.

Let’s consider single band first. If we pick some size m , and we have n items, each of them is hashed to some bit with probability $1/m$. The probability that the bit remains zero is: $p = (1 - 1/m)^n \approx e^{-n/m}$. If we set p , other parameters are uniquely determined, so we want to find p such that the probability of FP is minimized. If we set p , then $m \approx -n/\ln p$. Since all bands must fit in memory of M bits, we will use $k = \lfloor M/m \rfloor \approx - (M/n) \ln p$ bands.

On the other hand, we can use the same calculation to find the memory in bits needed for pushing the probability under some ϵ : $M = kn/\ln 2 \approx n \log(1/\epsilon) 1.44$

Alternatives:

- Single-table filters
 - instead of using separate arrays B_i , we can use multiple hash functions that point to bits in the shared table. Insertion of n elements now sets kn bits which are not necessarily distinct now. Therefore, we get probability of a bit $B[i]$ being set: $(1 - 1/m)^{nk}$ which is approximately $p = e^{-nk/m}$. Again the same performance is reached as in the previous case.
- Counting filters
 - Usual Bloom filters don’t support deletion of the elements. Counting filters use counters instead of bits in the arrays, and INSERT operation increments this counter, while DELETE decrements it. LOOKUP still works the same. Since counters have limited range, they can get stuck when they reach the maximum value. Too many stuck counters degrade the performance of the data structure, but it is possible to prove that this happens with small probability only. Note: I remember asking if rehashing everything after reaching a certain number of stuck counters was a good idea and he said yes, so maybe worth mentioning.

[Major] Define suffix arrays and LCP arrays. Describe and analyze algorithms for their construction.

Zhenya, last year lecture, lecture notes, good [video](#) about Kasai's algorithm, helpful [example](#) of building suffix array by doubling with bucketsort, [video](#) with a slightly different algorithm for building suffix array, but really helpful because the idea is the same. Yet [another video](#) on suffix array building. It does not go into bucket sorting but gives a nice example of how the doubling alg works.

NB: Look at the [notation](#) that is used for strings. Also, a [refresher for lexicographical order](#).

Suffix arrays are used to preprocess a text, so that we can find any pattern (string) in it.
Substrings $\Leftrightarrow$ prefixes of suffixes.

Definition: The **suffix array** for a string α of length n is a permutation S of the set $\{0, \dots, n\}$ such that $\alpha[S[i] :] < \alpha[S[i + 1] :]$ for all $0 \leq i < n$.

we need to store all suffixes of a string in lexicographic order
but direct storage of suffixes is space-inefficient
so it's better to store indices of starting positions of these suffixes in the string
suffixes are still in lexicographic order but we keep only 1 int for each suffix

Claim: The suffix array for a string of length n can be constructed in time $O(n)$.

Corollary: Using the suffix array for α , we can enumerate all occurrences of a substring β in time $O(|\beta| \log |\alpha| + p)$, where p is the number of occurrences reported. Only counting the occurrences costs $O(|\beta| \log |\alpha|)$ time.

binary search in suffix array for α with linear string comparison on each step

Definition: The **rank array** $R[0 \dots n]$ is the inverse permutation of S . That is, $R[i]$ tells how many suffixes of α are lexicographically smaller than $\alpha[i :]$.

also, $R[i]$ tells us at which position in lexicographic order $\alpha[i :]$ occurs.
 $\alpha[i :] < \alpha[j :] \Leftrightarrow R[i] < R[j]$

Note: The rank array can be trivially computed from the suffix array in time $O(n)$.

Definition: The **LCP array** $L[0 \dots n - 1]$ stores the length of the *longest common prefix* of each suffix and its lexicographic successor. That is, $L[i] = \text{LCP}(\alpha[S[i] :], \alpha[S[i + 1] :])$, where $\text{LCP}(\gamma, \delta)$ is the maximum k such that $\gamma[0 : k] = \delta[0 : k]$.

The LCP array is an array in which every element tracks how many characters two sorted adjacent suffixes have in common in their prefixes.

Claim: Given the suffix array, the LCP array can be constructed in time $O(n)$.

Example of Suffix array, Rank array, LCP array for the string `bananas`

		i	S(i)	R(i)	L(i)
	ϵ	0	7	4	0
	ananas	1	1	1	3
	anas	2	3	5	1
	as	3	5	2	0
	bananas	4	0	6	0
all suffixes starting with 'a'	nanas	5	2	3	2
	nas	6	4	7	0
	s	7	6	0	-

Construction of the LCP array: Kasai's algorithm

Let's use α_i to denote the i -th suffix of α in lexicographic order, that is $\alpha[S[i]:]$. So $\alpha_i = \alpha[S[i]:]$.

If we compute $L[i]$ explicitly, it would take $O(n^2)$ time in the worst case because every suffix should be compared with other suffixes. The idea of Kasai's algorithm is to **use already available LCP values for suffixes that are shortened by one character from the beginning**.

Consider two suffixes α_i and α_{i+1} adjacent in lexicographic order. Suppose that their LCP $k = L[i]$ is non-zero. Then $\alpha_i[1:]$ and $\alpha_{i+1}[1:]$ are also suffixes of α , equal to $\alpha_{i'}$ and $\alpha_{j'}$ for some $i' < j'$. Obviously, $LCP(\alpha_{i'}, \alpha_{j'}) = LCP(\alpha_i, \alpha_{i+1}) - 1 = k - 1$. However, this LCP is a minimum of the range $[i', j']$ in the array L , because $LCP(\alpha_{i'}, \alpha_{i'+1})$ (next suffix of α that we consider and its immediate neighbor) can be larger than $LCP(\alpha_{i'}, \alpha_{j'})$, so we must have $L[i'] \geq k - 1$.

[Kate] The easiest way to implement this logic is to go from the largest suffix of α to the smallest one and compare the current suffix with its neighbor in lexicographic order. For example, we have the longest suffix $\alpha[0:]$, we find its position in S (lexicographic order of suffixes) by taking $R[0]$ and find the next neighbor in S by taking $S[R[0] + 1]$ (let's call this neighbor β further in this passage). We find their LCP = k , then strip off one character from the suffix under consideration (so it will be $\alpha[1:]$). The logic is that $\beta[1:]$ comes after $\alpha[1:]$ in lexicographic order (the first char after LCP is larger for β because it was the next neighbor initially) but there might be other strings which are "lexicographically closer" to $\alpha[1:]$ and "interfere" between $\alpha[1:]$ and $\beta[1:]$ (see second pic in the example below). But all this range between $\alpha[1:]$ and $\beta[1:]$ will definitely share $k-1$ common characters because $\alpha[1:]$ and $\beta[1:]$ do. So we have to check LCP of $\alpha[1:]$ and its new next neighbor but **we can start the comparison from the $k-1$ th char** (if we index from 0).

Examples from the [video](#):

NB: here he takes a suffix and preceding neighbor (we take the next one)

i	$R[i]$	T_i	r	$L[r]$	$T_{L[r]}$	$LCP[r]$
→ 0	6 th	bananaban\$	0	9	\$	—
1	4 th	ananaban\$	1	5	aban\$	
2	9 th	nanaban\$	2	7	an\$	
3	3 th	anaban\$	3	3	anaban\$	
4	8 th	naban\$	4	1	ananaban\$	
5	1 th	aban\$	5	6	ban\$	
6	5 th	ban\$	→ 6	0	bananaban\$	3
7	2 th	an\$	7	8	n\$	
8	7 th	n\$	8	4	naban\$	
9	0 th	\$	9	2	nanaban\$	

i	$R[i]$	T_i	r	$L[r]$	$T_{L[r]}$	$LCP[r]$
0	6 th	bananaban\$	0	9	\$	—
→ 1	4 th	ananaban\$	1	5	aban\$	
2	9 th	nanaban\$	2	7	an\$	
3	3 th	anaban\$	3	3	anaban\$	
4	8 th	naban\$	→ 4	1	ananaban\$	3
5	1 th	aban\$	5	6	ban\$	
6	5 th	ban\$	6	0	bananaban\$	3
7	2 th	an\$	7	8	n\$	
8	7 th	n\$	8	4	naban\$	
9	0 th	\$	9	2	nanaban\$	

Pseudocode:

Algorithm BUILDLCP

Input: A string α of length n , its suffix array S and rank array R

1. $k \leftarrow 0$ $\triangleleft$ The LCP computed in previous step
2. For $p = 0, \dots, n - 1$: $\triangleleft$ Start of the current suffix in α
3. $k \leftarrow \max(k - 1, 0)$ $\triangleleft$ The next LCP is at least previous - 1
4. $i \leftarrow R[p]$ $\triangleleft$ Index of current suffix in sorted order
5. $q \leftarrow S[i + 1]$ $\triangleleft$ Start of the lexicographically next suffix in α
6. While $(p + k < n) \wedge (q + k < n) \wedge (\alpha[p + k] = \alpha[q + k])$:
7. $k \leftarrow k + 1$ $\triangleleft$ Increase k while characters match
8. $L[i] \leftarrow k$ $\triangleleft$ Record LCP in the array L

Output: LCP array L

Analysis

All operations outside the while loop take $O(n)$.

Let's use the potential approach to estimate the time for the while loop.

potential = $k \in [0, n]$, k changes by increasing by 1 (step 7) or decreasing by 1 (step 3). It means that the number of increases is smaller or equal to the number of decreases plus n . At the same time the number of decreases is not greater than n . So the total number of

increases is at most $2n$ and it means that time spent in the while loop is $O(n) \Rightarrow$ total time required in the worst case is $O(n)$.

potential = $k \in [0 - n]$
 k is changed by $++$, $--$
 $\Rightarrow \# \text{ increases} \leq \# \text{ decreases} + n \leq 2n$
 total time is $\Theta(n)$.

Construction of the suffix array by doubling

The idea is to sort suffixes by first k character for $k = 1, 2, 4, 8, \dots$

Definition: For any two strings γ and δ , we define comparison of prefixes of length k :

$\gamma =_k \delta$ if $\gamma[: k] = \delta[: k]$,

$\gamma \leq_k \delta$ if $\gamma[: k] \leq \delta[: k]$.

It's not a linear order, but quasiorder, because it's not antisymmetric.

We can define suffix array S_k which is a permutation of indices that sorts suffixes using k -order.

And also we can define rank array $R_k : R_k[i] = \#j : \alpha[S_k[j]] <_k \alpha[S_k[i]]$

This way we can compare the suffixes on a doubling stage when we already ordered suffixes using k -order:

$\alpha[i :] \leq_{2k} \alpha[j :] \Leftrightarrow \alpha[i :] \leq_k \alpha[j :] \vee (\alpha[i :] =_k \alpha[j :] \& \alpha[i + k :] \leq_k \alpha[j + k :]) \Leftrightarrow$

$\Leftrightarrow R_k[i] < R_k[j] \vee (R_k[i] = R_k[j] \& R_k[i + k] < R_k[j + k]) \Leftrightarrow$

$\Leftrightarrow (R_k[i], R_k[i + k]) \leq_{lex} (R_k[j], R_k[j + k])$ where $(R_k[t], R_k[s])$ is an ordered pair.

Every ordered pair is a pair of numbers. If we do a manual example, then we would first order suffixes by first k characters (first items in pairs), then we would order them by second k characters (second items in pairs).

NB If $i + k > n$ or $j + k > n$, then it can be replaced by an empty suffix.

Using general purpose sorting algorithm we can build S_{2k} out of S_k in $O(n \log n)$ time. So construction of the full suffix array will require $O(n \log^2 n)$ time.

We can improve time complexity by using [Bucketsort](#) to sort the pairs. As the pairs contain only numbers between 0 and n , we can sort in two passes with n buckets. This takes $O(n)$ time, so the whole algorithm runs in $O(n \log n)$ time. The first pass still remains $O(n \log n)$, unless we can assume that the alphabet is small enough to index buckets.

Applications:

Histogram of k -grams: we want to count occurrences of every substring of length k . Occurrences of every k -gram correspond to ranges of suffixes in their lexicographic order. These ranges can be easily identified, because we have $L[. . .] < k$ at their boundaries. We only have to be careful about suffixes shorter than k , which contain no k -gram.

The longest repeating substring of a string α : Consider two positions i and j in α . The length of the longest common substring starting at these positions is equal to the LCP of the suffixes $\alpha[i :]$ and $\alpha[j :]$, which is a minimum over some range in L . So it is always equal to some value in L . It is therefore sufficient to consider only pairs of suffixes adjacent in the lexicographic order, that is to find the maximum value in L .

[The longest common substring of two strings \$\alpha\$ and \$\beta\$](#)

[Minor] Show how to use suffix array and LCP array for finding the longest common substring of two strings.

Zhenya, last year lecture, lecture notes

Consider we have two strings α and β and we want to find the longest common substring of them.

- Build suffix array (S) and LCP array (L) for a string $\alpha\#\beta$ ($\#$ shouldn't occur in α or β), $\#$ is lexicographically smaller than any character in the strings.

We can notice that each suffix of $\alpha\#\beta$ corresponds to a suffix of either α or β .

Let's consider $\alpha = abyx$, $\beta = byc$. We can build a new string $\gamma = abyx\#byc$ and construct the arrays:

0	1	2	3	4	5	6	7
a	b	y	x	#	b	y	c

Suffixes in lex order	i	S[i]	L[i]
ϵ	0	8	0
#byc	1	4	0
abyx#byc	2	0	0
byc	3	5	2
byx#byc	4	1	0
c	5	7	0
x#byc	6	3	0
yc	7	6	1
yx#byc	8	2	-

- Find $\max \text{LCP}(\gamma[i:], \gamma[j:])$ such that $i < |\alpha|$, $j > |\alpha|$.

This problem can be reduced to the searching of maximum $L[t]$ such that $S[t] < |\alpha|$, $S[t + 1] > |\alpha|$ or vice versa.

It takes $O(n)$ time to find the longest common substring.

[Minor] Show how to perform 1-dimensional range queries on binary search trees.

Jacob, last year's and this year's notes

Intuitive explanation: A range query is basically a given range over the items in the space. The response should be all items that fall between that range. There are various ways of doing range queries over 1-dimensional structures, however, the more naive approaches usually do not perform very well. Here we will explore some naive and some more robust approaches to range queries in this dimensional space.

1. First approach: Just sort the array and then do binary search over the lower and upper bound of the range. This is very simple. But what is the time complexity?

Build: $O(n \log n)$; Space: $O(n)$; Query: $O(\log n)$

No real way of making this dynamic though. Also, no real way to expand this to higher dimensionalities.

2. Use binary search trees.

We first define that: For a node v : $\text{int}(v) := \{x \mid \text{searching for } x \text{ visits } v\}$. What does that mean? The interval for each node is just the **values** that visit the node whenever we are looking for them (these **values**). $\text{int}(\text{root}) = R$ where R is the real numbers. Additionally:

$\forall v$ node with key k and left child l and right child r we get that:

- $\text{int}(l) = \text{int}(v) \cap (-\infty, k)$
- $\text{int}(r) = \text{int}(v) \cap (k, \infty)$
- $\forall v$, $\text{int}(v)$ is an interval
- Everything below the subtree of the node v , lies within $\text{int}(v)$
- This also applies to external nodes

NOTE: the formal definition of interval just basically states that there are no gaps within the interval. $\forall x, y, z: x < y < z$: if $x, z \in S$ then $y \in S$ where S is an interval.

These properties allow us to traverse trees to find ranges. We basically check for overlaps between intervals and the query range. Let us define the alg formally:

Procedure RANGEQUERY(v, Q)

Input: a root of a subtree v , query range Q

1. If v is external, return.
2. If $\text{int}(v) \subseteq Q$, report the whole subtree rooted at v and return.
3. If $\text{key}(v) \in Q$, report the item at v .
4. $Q_\ell \leftarrow Q \cap \text{int}(\ell(v))$, $Q_r \leftarrow Q \cap \text{int}(r(v))$
5. If $Q_\ell \neq \emptyset$: call RANGEQUERY($\ell(v), Q_\ell$)
6. If $Q_r \neq \emptyset$: call RANGEQUERY($r(v), Q_r$)

But what is the performance of this approach?

Lemma: *RangeQuery* on a balanced BST visits $O(\log n)$ nodes/subtrees.

Proof: when we traverse the tree, our query range e.g. some $[\alpha, \beta]$, we will only traverse either to the right or to the left until we reach a node which is inside the query range. (We can call the nodes a and b the nodes that bound the range, and p their lowest common ancestor, meaning our node that is in the range). *The intuition here is that this is just like traversing a normal BST, we are cutting our number of nodes in half every time we go down a level. Now it remains to be seen what happens when we reach p .*

Note that e.g. the successor of b is outside the range (and thus any other nodes with higher keys than b). Same concept applies in reverse to a .

The way to prove the lemma is by covering what happens once we reach this node p :

Suppose we traverse the left child of p (right child will be symmetrical):

- The range overlaps with both of the children in the relevant subtree. In this case, we recurse to the left to keep “investigating” and we report the whole right side tree (this makes it so we don’t recurse superfluously). We also report the key of the root of the subtree.
- If the range only overlaps with the right side subtree, then we recurse over that subtree.

This guarantees that we only visit $O(\log n)$ nodes when searching. This is because, once we reached p , we still only recurse on one side of the tree, but we do it for both finding a and for finding b so we do something like “2 binary searches”, this is still $O(\log n)$.

Corollary: *RangeQuery* runs in $O(\log n + p)$ time, where p is the number of returned items.

What is the build time? $O(n \log n)$ just following algorithms to build perfect BSTs.

What about space? $O(n)$ as usual for BSTs.

NOTE: Some obvious extensions can be done to this approach, namely aggregations:

Counting queries: precompute subtree sizes. When adding a subtree, just add its size to a running total. $O(\log n)$ once more as time complexity.

Making this dynamic: just apply dynamic tree approaches seen at the beginning of the semester, like self balancing trees e.g. splay trees. You simply recompute the ranges when you rotate.

[Minor] Define k-d trees and show that they require $\Omega(\sqrt{n})$ time per 2-d range query.

Jacob, last year's and this year's notes

Resources: Martin Mareš video([11-geometric.mp4](#): 44:00)

Martin Mareš notes ([pg: 84-85](#))

Prereq: [range queries in 1-d data structures](#).

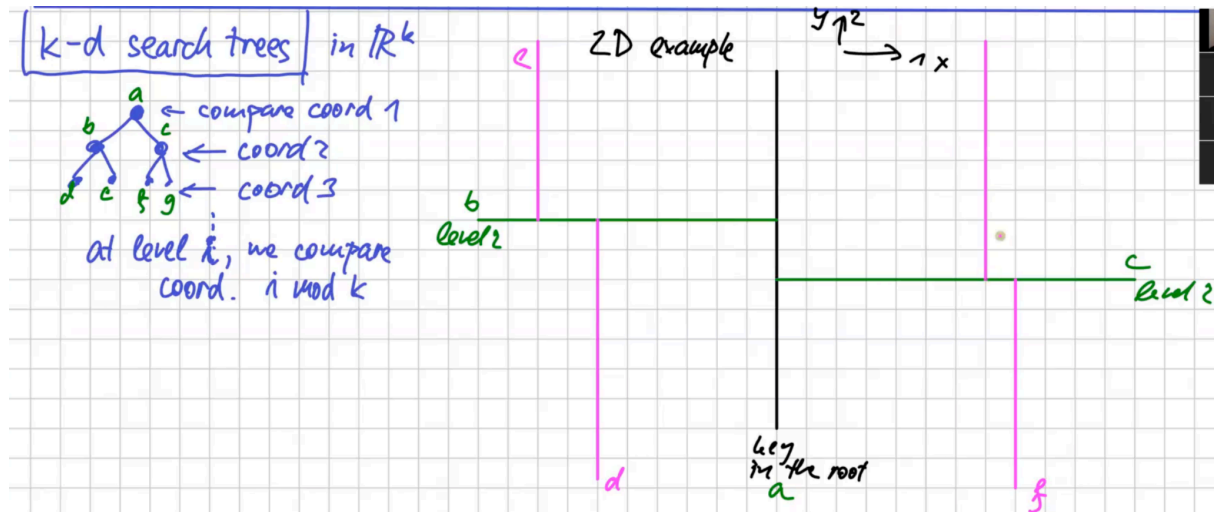
Intuitive explanation: This is an expansion of the previous approach to range queries. This time around, the keys are k-tuples of reals, hence the name k-dimensional trees. In a 2-d example, each node will split the space either vertically or horizontally. Meaning that every node can be assigned a 2D interval. This is great because the algorithm described before can be applied regardless of the dimensional space! Albeit with worse performance than for the 1-d space.

Definitions:

Let us have a BST where each node is a k-dimensional tuple $\in \mathbb{R}^k$. For the purposes of this section, let's restrict ourselves to 2-d spaces.

To every node of the tree, we can assign a 2-d range $int(v)$, which can be visualized as the regions below.

This means that every node splits the plane by alternating its dimension:



You can see that that node a split the plane in two horizontally, while b and c split each sub-plane vertically, and so on. However, note that they do not split it at the same point. **For now we will assume that no nodes cover the same points, i.e. each plane subdivision starts at different points.** This translates to no points sharing a coordinate. We will lift this restriction later.

The *RangeQuery* algorithm described in the previous section applies regardless of the dimensionality, so we can use it for this type of tree too. The correctness of the algorithm is based only on the nestedness of the tree and the ranges each node defines.

Build the 2-d search tree:

The key of the root node is the median of the x-coordinates. We then recurse on the two sub-spaces by finding medians but rotating the coordinates, meaning going to the next coordinate (and going back to the first if we reach the last one). Since the number of items on each recursion decreases we get that:

- Space $O(n)$
- Height of the tree is $O(\log n)$
- Build time $O(n \log n)$ because finding the median of m elements takes $O(m)$, so we spend $O(n)$ for each level.

These properties are as good as those we had for the 1-d approach.

Querying:

We have only stated that the *RangeQuery* algorithm works, but we haven't stated anything about its complexity. It turns out that it is slow.

To prove this, we will imagine a specific scenario with a **query range that is to the left of every point and covers all points in its height**. For this we pick the following points and query:

$$n := 2^t - 1$$

$$X := \{(i, i) \mid i \in \{1..n\}\}$$

$$Q := \{0\} \cdot \mathbb{R}$$

What does this mean? Well, we have our set of points $(1, 1), (2, 2), \dots, (n, n)$ in our tree of height t and a query that is the y-axis, (i.e. $x = 0$). This complies with what we stated above, our query is to the left of every point and it covers them all over the y-coordinate.

So, how does the *RangeQuery* alg look when approaching this?

At the root (looking at a x-coordinate split) we see that our query is to the left so we go to the left. But at the next node, the query overlaps both children so we recurse over both. We cannot just "automatically accept" some subtrees as we did in the 1-d case because we need to check the x-coordinates in them all. So on and so forth. So for every time we look at an x-coordinate node, we recurse only once to the left and for the others we recurse twice (on both children). So the number of leaves we visit doubles every other time.

$$\text{Number of leaves visited is } 2^{\text{height}/2} = 2^{t/2} = \sqrt{2^t} = \sqrt{n+1} \approx \sqrt{n}.$$

Quite terrible complexity! No nice $\log n$ complexity... This runs in $\Omega(\sqrt{n})$ time.

But what is worse, we can clearly see that **if we have even more dimensions, we will recurse on just 1 child even less often!** We will recurse twice for every dimension except for the one that aligns with the query ($1/d$ part of all recursion cases). Before, we alternated only between two dimensions, but now we alternate over d of them, and in our specific scenario, only one of them aligns with the query. So we recurse twice on $1 - 1/d$ of the coordinates, so our space doesn't just double every other node, but it grows at a $n^{1-1/d}$ rate.

This means that we get a complexity of: $O(n^{1-1/d})$ in R^d space! This is just the worst case, but still pretty bad. (You can see that it even approaches linear complexity if d approaches infinity hah).

However, the probability of this worst case scenario is probably very low, so this approach might still be useful but still. *Some side notes: if we insist on using $O(n)$ space, this is the best we can do. Secondly, it's hard to make this approach dynamic, in the sense that it is not straightforward to make this dynamic.*

Finally, what to do when points share coordinates? As promised, we can lift this condition and it won't affect the complexity.

We simply create a supplementary data structure along the lines of shared coordinates. It would be the equivalent of the 1-d structure. This would result in some nodes having a left child, a right child and a sort of middle child 1-d structure. We can simply refine what points fit the query range by using the alg from the 1-d structure.

NOTE: maybe someone could clarify here why checking the secondary DS results in only constant additions, i.e. the complexity does not change.

[Major] Define multi-dimensional range trees. Analyze time and space complexity of construction and range queries.

Jacob, last year's and this year's notes

Prereqs: [1-d range querying](#), also kinda [k-d trees](#)

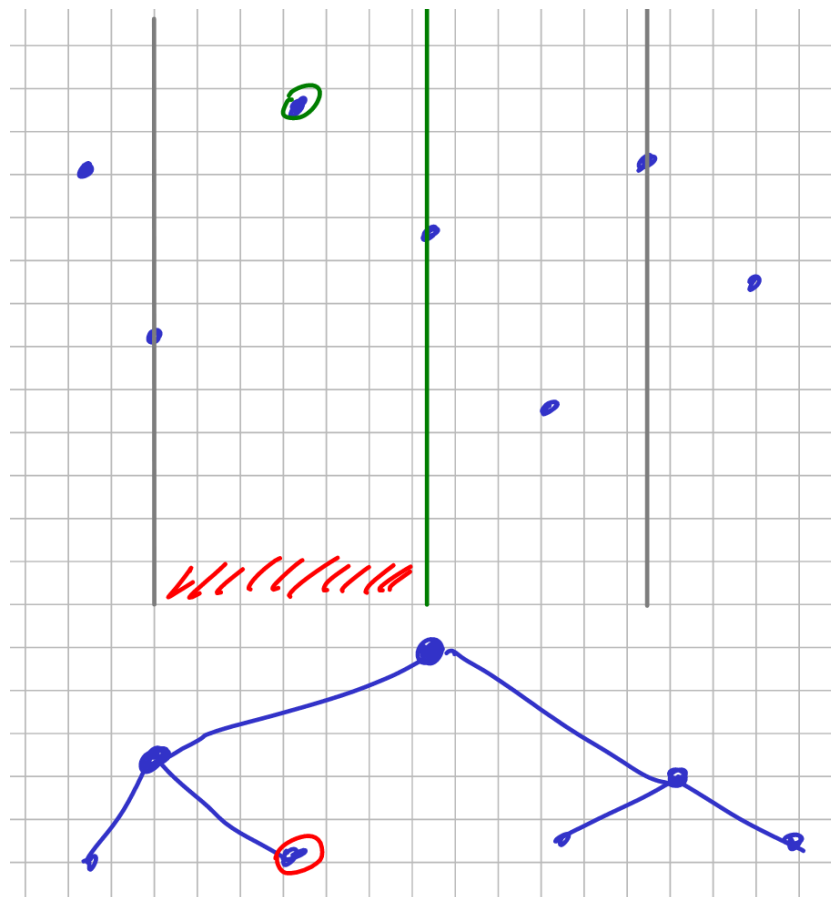
Intuitive explanation: Multi-dimensional data structures, also called geometric data structures, are useful for storing $\mathbb{R}^d$ dimensional objects. Multi-dimensional range trees are an instance of these that allow you to answer queries of a range type. These are an extension of the concepts shown in the 1-d range querying but in a different direction than k-d trees. Range trees have better time complexity than k-d trees, albeit at the cost of slightly worse space complexity.

Note: queries are usually written like so: $[a_1, b_1] \times [a_2, b_2]$.

2-d range trees:

To address this, we will first look at a specific case where $k = 2$, (the k in k -dimensional). We also re-use the assumption that no points share coordinates, meaning that e.g. points like $(2, 1)$ and $(5, 1)$ cannot be in our n points. This condition will be lifted soon.

For 2-d trees, we will have an *X-tree*. This tree will divide the plane in sub-planes per medians along the x-axis, recursively so. From these divisions we create a BST across this dimension as such:



The highlighted area (and all the way up) corresponds to a sub-plane defined by the node, and each of these are $(a_i, b_i) \times \mathbb{R}$ regions, i.e. bands (vertically unbounded rectangles). For every one of these bands we construct another tree, which we will call a **Y-Tree** and this tree will have all of the elements in an ordered manner according to their y coordinate now.

Space complexity: if the X-Tree BST is balanced, we know that we will have $O(n)$ bands, and thus every point will be in $O(\log n)$ bands. That will make our **total space complexity** $O(n \log n)$.

Build time complexity: we build this structure recursively after sorting all points.

1. Sort all points by the x -coordinate and by the y -coordinate. That results in two sorted lists of points. These lists are passed to all the recursions.
2. Now we can recurse. We build the X-tree and for every node in X-Tree we build the Y-Tree:
 - a. Find median m (in constant time cause we have the sorted list) and build X-Tree node
 - b. build Y-Tree for the node
 - c. recurse on each sub-band

Since the sorting (by both x and y) can be done in $O(n \log n)$ each and the recursing only depends on the sizes of the Y-Trees, we get that we can build the DS in $O(n \log n)$ time. Why? Each subproblem only takes linear time and we have $O(\log n)$ subproblems, so we get the aforementioned complexity.

Query: for $[a_1, b_1] \times [a_2, b_2]$. We spend $O(\log n)$ time on each subtree/band. Why? much like traversing a BST, it should take $O(\log n)$ to reach the desired node. And there are $O(\log n)$ bands, thus we get a complexity of $O(\log^2 n)$ for aggregated queries and $O(\log^2 n + p)$ for non-aggregated queries, where p is the number of returned items.

Lifting the shared coordinates restriction: We can simply just add an additional Y-Tree for every X-Tree node for points matching in a dimension. This only turns out to be constant times slower, thus not affecting our complexity.

Expanding this to k -d: We follow the same logic but instead of just adding one extra tree, we add trees for each extra dimension. Each of these will actually be a $k - 1$ -dimensional range tree.

Space complexity turns out to be: $O(n \log^{k-1} n)$ because we have each point $O(\log n)$ extra secondary trees.

Build time complexity: $O(n \log^{k-1} n)$ by induction from the previously analyzed build.

Query time complexity: $O(\log^k n + p)$ by induction as well. p need not be added if it's an aggregated query and no enumeration is needed.

[Minor] Show how to make multi-dimensional range trees dynamic (Insert suffices).

Kirill, lecture notes by Martin Mareš (1:22:20)

Let's consider a 2-dimensional range tree (see the [question above](#)). Briefly, it is the nested structure:

- the x-tree which is a range tree with all sorted x values. If we imagine the elements of the tree as dots in the 2-d plane, then x-tree slices it into "bands".
- set of y-trees for each x value from the x-tree. Each y-tree lives between two "bands" formed by the x-tree

Now, we might think of just using the dynamic instruments from the 1d BST, like splaying or lazy balancing. But if we do exactly the same here, it will be more expensive: whenever we change the x-tree, we change the "bands" and because of that all the related y-trees should change. At the same time, rebuilding the y-tree does not seem expensive.

So here is what we do: we will perform the lazy balancing strategy, for which in the 1-d trees the amortized cost was $O(\log(n))$ - we proved it there. Let's see how expensive it will be here.

When we do an Insert operation to a 2-d tree:

- the x-tree undergoes up to 1 insert
 - Insertion to the x-tree can cause rebalancing of the x-tree, which is more costly since rebuilding a subtree requires $O(n \log n)$ time instead of $O(n)$. So we have to spend $O(\log^2 n)$ amortized to pre-pay this cost.
 -
- y-trees undergo up to $\log(n)$ inserts (as each data point can participate in $O(\log(n))$ secondary trees)
 - Every insertion to a y-tree can cause rebalancing of that tree, which costs $O(\log n)$ amortized per tree. And there are up to $\log(n)$ trees. So $O(\log^2 n)$ altogether

We have therefore shown that saving $O(\log^2 n)$ time per Insert pays for all rebalancing of the constituent 1-d trees. The same argument applies to Delete.

The whole argument can be extended by induction to d-d trees: the dynamization of the partial rebuild would have $O(\log^d n)$ amortized cost

[Minor] Describe parallel (a,b)-trees with the use of locks.

Toni, lecture notes by Martin Mareš, lecture notes by Petr Gregor

Prerequisite

When we want to work with a data structure in a parallel way, we have to define where the locks will be. One lock per instance is sometimes not the best solution. If we have many small instances, individual locks are too memory costly. On the other hand, if we have a single big instance, locking it with one lock will become a performance bottleneck. So, how do we do that on a search tree such as (a,b)-tree?

Let's add a mutex to each node in the tree. Let's now consider operations Find, Insert and Delete on the tree.

Locking a path

When we want to do something with an item, we follow a path from the root and lock everything on the way.

- Find: just follows the path
- Insert: follows the path and adds a new leaf at the end (doesn't do balancing)
- Delete: follows the path and it finds either:
 - a node with at most 1 child: this one can be cut out
 - a node with 2 children: has to be replaced by its successor, but finding the successor just extends the path further down (probably to the right of the node?)
- We can even add rebalancing which goes back from a leaf to the top

Problem with this approach: every path contains the root! So we don't get any parallelization at all - everyone waits for the access to the root, and only one process can access the root at a time.

Locking a sliding window

If we consider only Find and Insert without balancing, there's no need to keep the nodes we already passed locked. It is sufficient to lock the current node only and its child (the one where we want to go to). This is a sliding window of size 2 over the path and locking only the nodes inside the window.

This approach is better, even though all paths start at the root, but the root is quickly unlocked and better parallelization is expected. When accessing random items, the paths will also be random, divergent, so there will not be much waiting.

Problem: How do we do Delete? How do we add balancing? When we do balancing, changes are propagated possibly to the root which has been maybe changed since we started following the path downwards because we don't hold a lock on it.

Top-down (a,b)-trees

The solution to our problem!

restriction: b must be at least $2a$

- Insert:

- hold a lock on the current node and its parent.
- In each step:
 - split the current node if it contains maximum number of keys (i.e. $b-1$)
 - for doing this, we only need a node and its parent
 - this guarantees that if a final node will be full after add-key, there will be space at its parent
 - unlock the parent
 - find the appropriate child and lock it
 - move there
- Delete
 - similar, but sometimes we also need to lock a sibling too
 - requires careful ordering of locks: first by level (depth), but at the same level we order the locks from left to right
 - possible problem: after examining the current node we decide to look at the left sibling (solution might be: lock the left sibling always before the current node) (another solution, we can lock children in arbitrary order if the parent is locked first [given in the exercise 2])
 - another problem: deletion of a key which is not in a leaf
 - Keys located on the higher levels cannot be removed directly — the internal node would lose one pointer and we would have a subtree left in our hands with no place to connect it to.
 - usually solved by replacing the key by its successor, but in a parallel setting while we are looking for the successor, we have to keep the current node locked. This can be slow, especially if the current node is a root so we are effectively locking the whole tree. In this case, we can keep the key in the tree and mark it as deleted

[Minor] Describe a lock-free stack including the memory management.

Kate, [lecture notes](#) + [lecture video](#) (starting from 42:57)

Useful [article](#)

Intuition: locks are useful but they have lots of problems so it would be nice to have some DS that have no need for locks. These will rely on atomic operations that will allow for checks before changes to the DS structure. We will also need to be careful about memory management here, and this is also addressed.

Kinda prerequisite: [locks](#)

but the reason for lock-free stacks is that locks have a lot of problems.

Local prerequisite - Atomic instructions:

guarantee atomicity and serve to achieve consistency in multiprocessing without locks
operate on cells of global memory called atomic registers

Read and write — the most basic guarantee is that an atomic register can be read or written as a whole. Concurrent accesses to the same atomic register are resolved in an unknown, but consistent order.

Exchange — exchange contents of an atomic register with a value in local memory. This is sufficient for implementing a mutex: an unlocked mutex will have a zero value. Locking the mutex will exchange the register with 1. If the original value is zero, we have succeeded. If it was 1, we retry. To unlock the mutex, we write a zero atomically.

Test and set bit — set a given bit of an atomic register and return its original value. This is another popular building block for locks.

Fetch and add — add a number to an atomic register and return its original value.

Compare and swap (CAS) — it is given an atomic register R and values *old* and *new*. If R equals *old*, it is changed to *new*. Otherwise R is not changed. In all cases, the original value of R is returned.

Load linked and store conditional (LL/SC) — a *linked load* is a normal atomic read, but the processor remembers the address and keeps monitoring if other processors access it. A later *store conditional* to the same address succeeds if no other processors wrote to this address. Otherwise it reports failure.

Lock-free stack:

represented as a linked list of nodes.

Each node will contain data of the item and an atomic pointer to the next item.

+ an atomic pointer to the head of the list (the most recent item in the stack).

PUSH: overall scheme, **not entirely correct** (explanation below)!!

Procedure PUSH

Input: A new node n containing the item to be pushed.

1. Repeat:
 2. $h \leftarrow \text{stack.head}$
 3. $n.\text{next} \leftarrow h$
 4. If $\text{CAS}(\text{stack.head}, h, n) = h$:
 5. Return.
- We have a problem here

In words:

1. We take the current head of a stack
2. We link it to the new head n from the input.
3. We call CAS, where stack.head is an atomic register, h is *old* and n is *new*. It must change the value from h to n in the register stack.head and return an old value h .

We need to use CAS because when we are at line 3, another process could've changed stack.head . CAS will fail if the value in stack.head register does not match *old*, which we passed. If it fails, we repeat until it succeeds.

POP: overall scheme, **not entirely correct** (explanation below)!!

Procedure POP

1. Repeat:
 2. $h \leftarrow \text{stack.head}$
 3. $s \leftarrow h.\text{next}$
 4. if $\text{CAS}(\text{stack.head}, h, s) = h$:
 5. Return h .
- We have a problem here

Output: A node removed from the top of the stack.

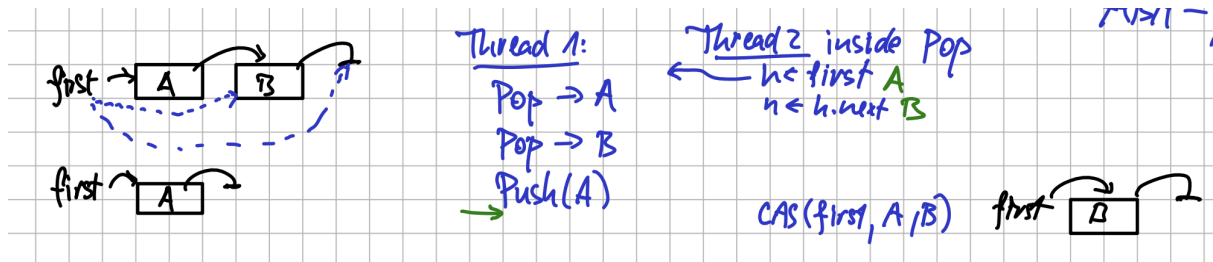
In words:

1. We take the current head of stack (h).
2. We take the next element of stack (s).
3. We call CAS, where stack.head is an atomic register, h is *old* and s is *new*. It must change the value from h to s in the register stack.head and return an old value h .

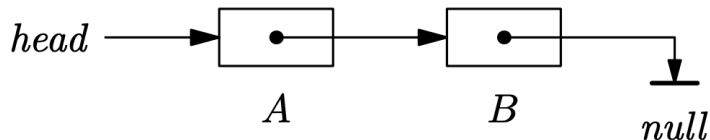
We need to use CAS because when we are at line 2, another process could've changed stack.head . CAS will fail if the value in stack.head register does not match *old*, which we passed. If it fails, we repeat until it succeeds.

Overall algorithm is okay, but there's a problem with CAS.

The ABA problem:



We have a stack:



- Process 2 is inside POP
 Process 2 takes `stack.head` and receives A
 Process 2 takes next element in the stack and receives B
 Process 2 loses control over CPU, it's taken by Process 1
- Then Process 1 starts execution
 Process 1 pops A
 Process 1 pops B
 The stack is empty
 Process 1 pushes A (the same object) in the stack
 A is the `stack.head` now
- Process 2 now has in its variables A - the element to pop, and B - next `stack.head`
 makes CAS, with A being *old* and B being *new*.
 CAS succeeds because A is really the `stack.head` (Process 1 pushed it back), but it replaces A with B in the `stack.head` register, but B was not in the stack (Process 1 popped it in its second step)

$\Rightarrow$ we have a "ghost" object which reappeared after its deletion.

Possible solutions:

- Use LL/SC instead of CAS.

If step 2 of Pop is a LL and step 4 is a SC, the SC fails even if the head changed from A to B and back in the meantime.

because the register `stack.head` has been changed by Process 1 and LL/SC keeps track of it.

drawback: LL/SC instruction is rarely implemented in the hardware.

- Have the machine support a double-CAS (alias CAS2), which is a CAS on a pair of atomic registers simultaneously.

We can replace the CAS in step 4 by $\text{CAS2}(\langle \text{stack.head}, h.\text{next} \rangle, \langle h, n \rangle, \langle n, n \rangle)$,

CAS2 changes `stack.head` from h to n, as usual CAS

second variable is required so that CAS2 fails if the head node was re-connected and it has a different successor now

in our ABA example, A was a head again but had another successor (*null* instead of B)

drawback: no current processor supports CAS2.

3. Have the machine support a wide CAS (WCAS), also called double CAS (DCAS). It is a CAS2 on two variables which are adjacent in memory.

This can be used for versioning of pointers: each pointer will be accompanied by an integer version, which will be incremented every time the pointer is changed.

Step 2 becomes $\langle h, v \rangle \leftarrow \langle \text{stack.head}, \text{stack.head_version} \rangle$ and the test in step 4 becomes If $\text{DCAS}(\langle \text{stack.head}, \text{stack.head_version} \rangle, \langle h, v \rangle, \langle n, v + 1 \rangle) = \langle h, v \rangle$.

first variable - head - DCAS changes `stack.head` from `h` to `n`, as usual CAS

second variable - version - DCAS fails if versions do not match, and if they do, version is incremented by 1.

no drawbacks! Unlike general CAS2, DCAS is often supported by hardware.

4. Avoid recycling of nodes: for every Push, allocate a new node.

In our example: make a copy of A and push it instead of the original.

Memory allocation

Why bother?

Locks: free memory used by a node after Pop.

Lock-free: even though the node is already popped, other processes can still access it before Pop finishes, for example getting a pointer to this node. Later, their CAS will reveal that the node is no longer present. But before calling CAS they might access invalid memory and crash.

Solution: collect all unused nodes in a free list — an atomic list managed like our lock-free stack — and free them after we make sure that no processes reference them.

How to do that?

1. Global synchronization — from time to time, we synchronize all processes at a point where they hold no pointers of their own, and free up all chunks from the free list.

This is simple, but in many cases the synchronization points are hard to find.

2. Reference counting — in every node, we keep an atomic counter of references to the node (i.e., local variables which point to the node). Again, we keep a free list. From time to time, we scan the free list and free up all nodes with zero references.

Amortized time of scan - $O(n \text{ freed nodes})$

We have P processes and each of them holds at most R references

⇒ there are at most RP references at any given moment.

we start the scan once the list accumulates at least $2RP$ items

⇒ we always free up at least a half of the nodes = RP .

⇒ time spent on scanning can be charged on the freed nodes, each node paying $O(1)$ time.

We add reference count to Pop operation:

Procedure POPREFCNT

1. Repeat:
2. $h \leftarrow \text{stack.head}$
3. Increment $h.\text{ref_cnt}$.
4. If $h \neq \text{stack.head}$:
5. Decrement $h.\text{ref_cnt}$ and retry the loop.
6. $s \leftarrow h.\text{next}$
7. if $\text{CAS}(\text{stack.head}, h, s) = h$:
8. Decrement $h.\text{ref_cnt}$ and return h .
9. Decrement $h.\text{ref_cnt}$.

Output: A node removed from the top of the stack.

In words:

1. We get a pointer to `stack.head` and increment its ref counter.
2. If `stack.head` has changed while we were incrementing the counter, we cancel our increment and retry the whole loop.
3. Proceed with usual Pop.
4. If CAS succeeded, h is only “ours” (it’s already removed from stack) and we can safely decrement its ref counter and return it.
5. If CAS failed (`stack.head` changed), we also decrement the ref counter of h (i.e. cancel our initial increment) and retry.

We don’t need to include reference count to Push because it references only new node, which is available only to us, other processes do not know about it.

3. Hazard pointers - use global array and for each thread, keep a fixed-size block of pointers to accessed items (aka “hazard pointers”).

Procedure POPHP

1. Repeat:
2. $h \leftarrow \text{stack.head}$
3. $hp \leftarrow h$ $\triangleleft$ One of our hazard pointers
4. If $h \neq \text{stack.head}$:
5. Retry the loop. $\triangleleft$ No need to reset hp here
6. $s \leftarrow h.\text{next}$
7. if $\text{CAS}(\text{stack.head}, h, s) = h$:
8. Let $hp \leftarrow \emptyset$ and return h .

Output: A node removed from the top of the stack.

In words:

1. We get a pointer to `stack.head` and we set a hazard pointer to it.
2. If `stack.head` has changed while we were setting the hazard pointer, we retry the whole loop.

[thought] we don’t need to reset hp neither here nor after CAS failure because we will overwrite it anyway during the retry

3. Proceed with Pop
 4. If CAS is successful, erase the hazard pointer and return h .
 5. If CAS fails, just repeat.
- As in case of reference counters, Push doesn't need to be modified with hazard pointers.

Deallocating memory:

We have a free list which contains all presumably unused nodes.

To match the free list and the hazard pointer array:

- we take a snapshot of the hazard pointer array
- we build a static data structure for the set of hazard pointers
 - in the simple case, sorted array with binary search
- for each node in the free list, we query the data structure to see if the node is still being accessed

Hazard pointer array snapshot is up-to-date and relevant. Proof:

Before a node enters the free list, it was disconnected from the data structure by a PopHP.

If any concurrent PopHP reached step 6 with h pointing to the node, the node was not in the free list yet.

⇒ the hazard pointer in the concurrent PopHP is set before the node enters the free list.

⇒ the snapshot taken when freeing memory includes all relevant hazard pointers.

Hard-to-remember things for last minute revision

[Minor] Describe a flexible array with growing and shrinking. Analyze its amortized complexity.

growing - aggregation method - evaluate overall time for n appends

total reallocs: $2^1 + 2^2 + \dots + 2^k = 2^{k+1} - 2 \in \Theta(n)$ and $2^k \in \Theta(n)$

shrinking: accounting method - redistribute time between operations

divide seq of operations into blocks, each block ends with realloc

$\Theta(C)$ operations in the block and reallocation takes $\Theta(C)$ time

[Minor] Define the lazily balanced trees BB[alpha]. Analyze their amortized complexity.

Definition: A node v is *in balance*, if for each its child c , we have $s(c) \leq \frac{2}{3} \cdot s(v)$

Lemma: The height of a balanced tree with n nodes is $O(\log n)$

FIND: $O(\log n)$

INSERT: amortized $O(\log n)$, needs proving

potential: sum among node contribs, the difference of sizes of its left and right child

The whole potential decreases by at least $\frac{1}{3} \cdot s(v)$ and real cost is $\Theta(s(v))$

[Major] Define the Splay tree. State and prove the theorem on amortized complexity of the Splay operation.

Theorem: The amortized cost of $\text{Splay}(x)$ is at most $3(r'(x) - r(x)) + 1$ which is $O(\log n)$, calc amortized cost for each operation, sum up

[Minor] Design operations Find, Insert, and Delete on a Splay tree. Analyze their amortized complexity.

FIND $O(\log n)$ - going to depth d , amortized by Splay which is $O(\log n)$

INSERT:

Lemma: Adding a leaf to the tree increases potential by $O(\log n)$

$$\Delta\Phi = r'(v_{t+1}) + \sum_{i=1}^t (r'(v_i) - r(v_i)).$$

bound from above, telescope to $\Delta\Phi \leq r'(v_1) - r(v_t)$ and this is logarithmic

DELETE - remove node, replace with child or successor, splay parent. $O(\log n)$

[Major] Define the (a,b)-tree. Describe operations Find, Insert, and Delete. Analyze their complexity in the worst case.

Lemma: The height of an (a,b)-tree with n keys lies between $\log_b(n + 1)$ and $1 + \log_a(\frac{n+1}{2})$

Corollary: The height is $\Omega(\log_b n)$ and $O(\log_a n)$.

FIND: $O(\frac{\log b}{\log a} \log n)$

INSERT: $O(b^{\frac{\log n}{\log a}})$

DELETE: $O(b^{\frac{\log n}{\log a}})$

[Minor] State and prove the theorem on amortized complexity on Insert and Delete on $(a, 2a-1)$ -trees and $(a, 2a)$ -trees.

Theorem: A sequence of m Inserts on an initially empty (a, b) -tree performs $O(m)$ node modifications.

total splits $\leq$ # final internal nodes $\leq m$

Theorem: A sequence of m Inserts and Deletes on an initially empty $(a, 2a)$ -tree performs $O(m)$ node modifications.

$$\Phi = \sum_{v \text{ all nodes}} f(\# \text{keys in } v)$$

function reqs:

for constant change: $|f(i) - f(i + 1)| \leq c$

free splits: $f(2a) \geq f(a) + f(a - 1) + c + 1$

free merges: $f(a - 2) + f(a - 1) \geq f(2a - 2) + c + 1$

k	$a - 2$	$a - 1$	a	$\dots$	$2a - 2$	$2a - 1$	$2a$
The function $f(k)$	2	1	0	$\dots$	0	2	4

Amortized cost of Insert equals $O(c)$ [initial insertion, req 1 above] + 0 [per split].

Amortized cost of Delete equals $O(c)$ [initial delete] + 0 [per merge] + $O(c)$ [stealing key]

[Minor] Analyze k -way Mergesort in the cache-aware model. Which is the optimum value of k ?

Time complexity: $\Theta(N \log N)$ for any K

I/O complexity:

$O(N / B \cdot \log N / \log K + 1)$

Optimal K : $K = M / 2B$

if K is optimal, then $O(N / B \cdot \log N / \log (M / B) + 1)$

[Major] Formulate a cache-oblivious algorithm for transposition of a square matrix. Analyze its time complexity and I/O complexity.

Algo - divide and conquer

Time complexity: tree has at most $4^{\log N} = N^2$ leaves, processing only in leaves $\Rightarrow O(N^2)$

I/O complexity:

recurse until fits into cache, fits when $N / 2^i \in \Theta(B)$

then like in cache-aware with $d = B \Rightarrow$ complexity is $O(N^2 / B + 1)$

Tall-cache property: $M \in \Omega(B^2)$

[Minor] State and prove the Sleator-Tarjan theorem on competitiveness of LRU.

Theorem:

For every $M_{\text{LRU}} > M_{\text{OPT}} \geq 1$ and every request sequence, we have:

$$C_{\text{LRU}} \leq \frac{M_{\text{LRU}}}{M_{\text{LRU}} - M_{\text{OPT}}} \cdot C_{\text{OPT}} + M_{\text{OPT}}.$$

Reminder: M is the number of slots in the cache.

Also: we give larger size to LRU than to OPT because LRU is not k -competitive with the same cache size.

Split the problem in epochs, E_0 has the “remaining” cache misses.

consider cases: E_i with diff misses, E_i with same miss twice, E_0 with empty cache, E_1 with full OPT cache

[Minor] Define c -universal and k -independent systems of hash functions. State and prove the theorem on the expected chain length in hashing with chains.

Let $\mathcal{H}$ be a family of functions $\mathcal{U} \rightarrow [m]$ we say that the family is c -universal for some $c > 0$ iff $\forall x, y \in \mathcal{U}, x \neq y: \Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{c}{m}$

Theorem: Let $\mathcal{H}$ be a c -universal family of functions from $\mathcal{U}$ to $[m]$, $X = \{x_1, \dots, x_n\} \subseteq \mathcal{U}$ a set of numbers stored in the DS and $y \in \mathcal{U} \setminus X$ an item not stored in the DS, then we have:

$$\mathbb{E}_{h \in \mathcal{H}}[\#i: h(x_i) = h(y)] \leq \frac{cn}{m}$$

Proof: by expectation over indicators

Corollary on the complexity of hashing with chaining: FIND and others are $O(1)$ if $m = \Omega(n)$

Let $\mathcal{H}$ be a family of hashing functions from $\mathcal{U}$ to $[m]$. The family is (k, c) -independent for a k , $1 \leq k \leq |\mathcal{U}|$ and a $c > 1$ if and only if for every tuple $x_1, \dots, x_k$ of **distinct** elements of the universe $\mathcal{U}$ and every k -tuple $a_1, \dots, a_k$ from $[m]$ the following holds

$$\Pr_{h \in \mathcal{H}}[h(x_1) = a_1 \wedge \dots \wedge h(x_k) = a_k] \leq \frac{c}{m^k}$$

[Minor] Describe a system of linear hash functions. Prove that it is a 2-independent system from $\mathbb{Z}_p$ to $[m]$.

$\mathcal{L} = \{h_{a,b} \mid a, b \in \mathbb{Z}_p\}$ from $\mathbb{Z}_p$ to $[m]$ is a system of linear hash functions for prime p and $m \leq p$, where $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$.

Theorem: The family $\mathcal{L}$ is (2,4)-independent.

(a, b) lead to unique solution

prob of congruence mod m - consider intervals - $(p/m) / p$

$$\text{strange rounding: } \frac{\lfloor p/m \rfloor}{p} \leq \frac{p+m-1}{mp} \leq \frac{2p}{mp} = \frac{2}{m}$$

[Minor] Construct a k -independent system of hash functions from $\mathbb{Z}_p$ to $[m]$.

$$\mathcal{P}_k = \{h_{\vec{t}} | \vec{t} \in \mathbb{Z}_p^k\} \text{ from } \mathbb{Z}_p \text{ to } \mathbb{Z}_p \text{ where } h_{\vec{t}}(x) = \sum_{i=0}^{k-1} t_i x^i \text{ where } \vec{t} \text{ is a } k\text{-tuple indexed from } 0$$

Lemma: Family $\mathcal{P}$ is $(k, 1)$ -independent.

$\forall i: h_{\vec{t}}(x_i) = a_i$ determines a unique polynomial that fits the equation and p^k total options for $\mathcal{P} \bmod m$ apply lemma K for $p \geq 2km$ and get $(k, 2)$ -independent family

[Minor] Construct a 2-independent system of hash functions for hashing of strings of length at most L over an alphabet $[a]$ to a set of buckets $[m]$.

rolling hashing functions:

$$\mathcal{R} = \{h_a | a \in \mathbb{Z}_p\} \text{ from } \mathbb{Z}_p^d \text{ to } \mathbb{Z}_p \text{ where } h_a(x) = \sum_{i=0}^{d-1} x_{i+1} \cdot a^i = x_1 + x_2 a + x_3 a^2 + \dots + x_d a^{d-1}$$

Theorem: The family $\mathcal{R}$ is d -universal.

collision: $\sum_i (\vec{x} - \vec{y})_{i+1} \cdot a^i = 0$, a^i is a root of the $(\vec{x} - \vec{y})$ polynomial, only d roots possible

composition lemma: composition is $(2, c')$ -independent for $c' = (cm/r + 1)d$.

$\mathcal{R} \circ \mathcal{L}$, $(2, 5/2)$ -independent

Rabin-Karp - sliding windows, $H_{j+1} = aH_j - \sigma_j a^d + \sigma_{j+d}$.

String hashing - pad seqs to L with zeroes and choose $p \gg k$

[Minor] Describe a system of hash functions based on scalar products. Prove that it is a 1-universal system from $\mathbb{Z}_p^k$ to $\mathbb{Z}_p$.

family of scalar product hash functions

$$S = \{h_{\vec{t}} | \vec{t} \in \mathbb{Z}_p^k\} \text{ from } \mathbb{Z}_p^k \text{ to } \mathbb{Z}_p, \text{ where } h_{\vec{t}}(\vec{x}) = \vec{x} \cdot \vec{t}.$$

Theorem: S is 1-universal

$$\Pr_{h \in S}[h_{\vec{t}}(\vec{x}) = h_{\vec{t}}(\vec{y})] = \Pr[\vec{x} \cdot \vec{t} = \vec{y} \cdot \vec{t}] = \Pr[(\vec{x} - \vec{y}) \cdot \vec{t} = 0] = \Pr[(x_k - y_k) \cdot t_k = -\sum_{i=1}^{k-1} (x_i - y_i)t_i]$$

a unique solution of t_k .

for mod m : composition, $S \circ \mathcal{L}$

[Minor] Describe the cuckoo hashing and state the theorem on its complexity (without proof).

Theorem: Let $\epsilon > 0$ be a fixed constant. Suppose that $m \geq (2 + \epsilon)n$, insertion timeout is set to $\lceil 6 \log n \rceil$, and f, g chosen at random from a $\lceil 6 \log n \rceil$ -independent family. Then the expected time complexity of INSERT is $O(1)$, where the constant in O depends on ϵ .

[Major] Describe and analyze hashing with linear probing

Theorem: Let $m \geq 3n$ be a power of 2, $h: U \rightarrow [m]$ totally random function, $x \in U$, then the expected number of probes in FIND is $O(1)$.

Chernoff bound to the right tail: $\Pr[X > \mu c] \leq \left(\frac{e^{c-1}}{c}\right)^\mu$

Definition: A **block** B is **critical** if more than $2/3 \cdot 2^t$ items are hashed to B .

Lemma 1: Let B be a block of size 2^t . Then $\Pr_h[B \text{ is critical}] \leq (e/4)^{2^{t/3}}$.

Lemma 2: Let R be a run of length 2^{l+2} , $B_0, \dots, B_3$ as the first blocks in the run of size 2^l intersecting R . Then at least one of these 4 blocks is critical.

Lemma 3: Let R be a run containing $h(x)$ and $|R| \in [2^{l+2}, 2^{l+3})$, then at least one of the following blocks is critical:

1. Either the block containing $h(x)$
2. One of the 8 blocks before that block
3. One of the 3 blocks after that block

Corollary: If we have a run R containing $h(x)$ then $\Pr[|R| \in [2^{l+2}, 2^{l+3})] \leq 12 \cdot q^{2^l}$.

Concluding the proof: find the expected length of a run.

[Minor] Describe and analyze the Bloom filter

use hash func from 1-universal family for indexing bit array

prob of FP (collision) $1/m$, prob of at least 1 FP among $x_1 \dots x_n$: n/m

push error probability under some $\epsilon > 0$: $m = \text{ceil}(n/\epsilon)$

[Major] Define suffix arrays and LCP arrays. Describe and analyze algorithms for their construction.

Algorithm BUILDLCP

Input: A string α of length n , its suffix array S and rank array R

1. $k \leftarrow 0$ $\triangleleft$ The LCP computed in previous step
2. For $p = 0, \dots, n-1$: $\triangleleft$ Start of the current suffix in α
3. $k \leftarrow \max(k-1, 0)$ $\triangleleft$ The next LCP is at least previous - 1
4. $i \leftarrow R[p]$ $\triangleleft$ Index of current suffix in sorted order
5. $q \leftarrow S[i+1]$ $\triangleleft$ Start of the lexicographically next suffix in α
6. While $(p+k < n) \wedge (q+k < n) \wedge (\alpha[p+k] = \alpha[q+k])$:
7. $k \leftarrow k+1$ $\triangleleft$ Increase k while characters match
8. $L[i] \leftarrow k$ $\triangleleft$ Record LCP in the array L

Output: LCP array L

Time complexity: $O(n)$

construction of suffix array by doubling:

$$\begin{aligned} \alpha[i:] \leq_{2k} \alpha[j:] &\Leftrightarrow \alpha[i:] \leq_k \alpha[j:] \vee (\alpha[i:] =_k \alpha[j:] \& \alpha[i+k:] \leq_k \alpha[j+k:]) \Leftrightarrow \\ &\Leftrightarrow R_k[i] < R_k[j] \vee (R_k[i] = R_k[j] \& R_k[i+k] < R_k[j+k]) \Leftrightarrow \end{aligned}$$

$\Leftrightarrow (R_k[i], R_k[i + k]) \leq_{lex} (R_k[j], R_k[j + k])$ where $(R_k[t], R_k[s])$ is an ordered pair.

Time complexity: $O(n \log^2 n)$, with Bucketsort $O(n \log n)$

[Minor] Show how to use suffix array and LCP array for finding the longest common substring of two strings.

S and L for $\alpha\#\beta$

maximum $L[t]$ such that $S[t] < |\alpha|$, $S[t + 1] > |\alpha|$ or vice versa

[Minor] Show how to perform 1-dimensional range queries on binary search trees.

$int(v) := \{x \mid \text{searching for } x \text{ visits } v\}$

Procedure RANGEQUERY(v, Q)

Input: a root of a subtree v , query range Q

1. If v is external, return.
2. If $int(v) \subseteq Q$, report the whole subtree rooted at v and return.
3. If $key(v) \in Q$, report the item at v .
4. $Q_\ell \leftarrow Q \cap int(\ell(v))$, $Q_r \leftarrow Q \cap int(r(v))$
5. If $Q_\ell \neq \emptyset$: call RANGEQUERY($\ell(v), Q_\ell$)
6. If $Q_r \neq \emptyset$: call RANGEQUERY($r(v), Q_r$)

Lemma: RangeQuery on a balanced BST visits $O(\log n)$ nodes/subtrees.

once we reached p , we still only recurse on one side of the tree, but we do it for both finding a and for finding b so we do something like “2 binary searches”, this is still $O(\log n)$.

[Minor] Define k-d trees and show that they require $\Omega(\sqrt{n})$ time per 2-d range query.

On each level, split and recurse on left and right subtrees. $O(n)$ per level, $\log n$ levels $\rightarrow O(n \log n)$ build, space $O(n)$.

Consider diagonal points $\{(i, i) \mid 1 \leq i \leq n\}$ for $n = 2^t - 1$. RangeQuery($\{0\} \times \mathbb{R}$) goes left on x-coordinate repeats, left and right on y-coordinate repeats. Visited nodes doubles at every other level $\rightarrow 2^{t/2} \approx \sqrt{n}$ nodes at $t \rightarrow \Omega(\sqrt{n})$ time.c

[Major] Define multi-dimensional range trees. Analyze time and space complexity of construction and range queries.

Space complexity: $O(n \log^{k-1} n)$

Build time complexity: $O(n \log^{k-1} n)$

Query time complexity: $O(\log^k n + p)$

[Minor] Describe parallel (a,b)-trees with the use of locks.

top-down (a,b) trees

lock current node and parent

pre-emptive split

pre-emptive merge or borrowing from a sibling + tombstones for internal nodes

[Minor] Describe a lock-free stack including the memory management.

TODO